

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

REPORTE

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar.
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar.
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencko@fi.uba.ar.
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar.

Fecha de entrega: 01/05/25

ÍNDICE

Ejercicio 1: Análisis exploratorio	3
Introducción a la organización del trabajo	3
Breve descripción y análisis del dataset	3
Supuestos e hipótesis	6
Análisis más profundo de los registros	7
Datos faltantes y eliminación de columnas	7
Correlación y covarianza	8
Generación de nuevos features	9
Valores atípicos	10
Preguntas de investigación	12
Supuestos	16
 Ejercicio 2: Modelos de Clasificación Binaria	 21
Breve descripción y análisis del dataset	21
Entrenamiento y predicción	22
Modelo 1: Árbol de decisión	22
Modelo 2: Random Forest	25
Modelo 3: Voting Classifier	28
Cuadro de resultados	29
Cuadro de tiempo dedicado	33
 Ejercicio 3: Regresión	 35
Breve descripción y análisis del dataset	35
Entrenamiento y predicción	37
Modelo 1: Regresión Lineal Múltiple	38
Modelo 2: XGBoost	39
Modelo 3: K-Nearest Neighbors (K-NN)	39
Cuadro de resultados	41
Cuadro de tiempo dedicado	43
 Ejercicio 4: Clustering	 45
Breve descripción y análisis del dataset	45
Análisis de la tendencia al clustering	46
Cuadro de tiempo dedicado	50

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

Ejercicio 1: *Análisis Exploratorio de Datos*

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar.
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar.
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencho@fi.uba.ar.
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar.

Fecha de entrega: 01/05/25

Ejercicio 1: Análisis exploratorio

Introducción a la organización del trabajo

Optamos por vincular Colab con nuestro repositorio de Github y creamos un servidor de Discord para poder comunicarnos más cómodamente. Hicimos llamadas regularmente para ayudarnos entre nosotros, trabajar en conjunto y debatir distintas ideas para el trabajo. Además, creamos una hoja de [Excel](#) para la división de tareas y una [planilla](#) con las funciones vistas en clase.

Breve descripción y análisis del dataset

Meses asignados: Julio / Agosto / Septiembre.

El Dataset contiene información sobre los viajes de los Taxis amarillos de Nueva York. Inicialmente la información estaba dividida por mes en archivos de formato .parquet, pero decidimos unirla en un mismo DataFrame concatenando por filas. Finalmente, la cantidad de registros inicial fue de 9.689.116 y la cantidad de columnas inicial fue 19.

El siguiente paso fue establecer qué tipo de variable era cada columna. Las dividimos en Cuantitativas y Cualitativas, que a su vez clasificamos en continuas y discretas y en nominales y ordinales, respectivamente. El criterio establecido para esta clasificación fue el propuesto por los apuntes de la materia¹, aunque también consultamos fuentes externas². Además, en el Colab traducimos el [Data Dictionary – Yellow Taxi Trip Records](#) proporcionado por la cátedra para comprender mejor qué significaba cada columna. La división final fue la siguiente:

¹ Sin autor. (2025, marzo 25). Tipos de variables. FIUBA, Cátedra Dr. Ing. Rodríguez. Recuperado de: https://docs.google.com/document/d/1e8hDYVVx1PZScIsttjIGGE0Bci1reZH8kFx8CyHPgl4/edit?usp=drive_link

² Julca, J. E. (2025, febrero 22). Tipos de variables estadísticas: Cuantitativas y Cualitativas. Escuela de Investigación. Recuperado de: <https://escueladeinvestigacion.com/2025/02/22/tipos-de-variables-estadisticas-cuantitativas-y-cualitativas/>

Sin autor. (2022, junio 7). ¿Qué es una variable discreta? (ejemplos). Probabilidad y estadística. Recuperado de: <https://www.probabilidadyestadistica.net/variable-discreta/>

Tipo de variable	Cuantitativas	Tipo de variable	Cualitativas
Continua	tpep_pickup_datetime, tpep_dropoff_datetime, trip_distance, fare_amount, extra, mta_tax, tip_amount, tolls_amount, total_amount, congestion_surcharge.	Nominal	VendorID, store_and_fwd_flag, PULocationID, DOLocationID, payment_type, RatecodeID
Discreta	passenger_count, improvement_surcharge, Airport_fee	Ordinal	—

En particular:

Cuantitativas

Para establecer qué variables pertenecían a la categoría de cuantitativas, tomamos la definición propuesta por la cátedra: “...entre dos valores de la variable existen infinitos valores intermedios...”

- **tpep_pickup_datetime:** La fecha y hora en que el taxímetro fue activado. **Tipo:** cuantitativa **continua**, ya que, al ser una fecha representa una medición en el tiempo que puede tomar infinitos valores dentro de un rango.
- **tpep_dropoff_datetime:** La fecha y hora en el que el taxímetro fue desactivado. **Tipo:** cuantitativa **continua**, ya que, al ser una fecha representa una medición en el tiempo que puede tomar infinitos valores dentro de un rango.
- **passenger_count:** El número de pasajeros en el vehículo, es un valor ingresado por el conductor. **Tipo:** cuantitativa **discreta** porque representa una cantidad que es contable y solo puede tomar valores enteros.
- **trip_distance:** La distancia del viaje transcurrida en millas reportada por el taxímetro. **Tipo:** cuantitativa **continua** representa una medición de distancia (en millas), y puede tomar infinitos valores dentro de un intervalo.
- **fare_amount:** La tarifa calculada por el taxímetro basada en el tiempo y la distancia. **Tipo:** cuantitativa **continua** porque representa dinero y admite decimales entre dos valores.
- **extra:** Extras y recargos varios. Actualmente, esto solo incluye los recargos de USD 0.50 y USD 1 por hora punta y durante la noche. **Tipo:** cuantitativa

continua porque el recargo se va sumando, lo que genera que dentro de extra haya números no enteros y no hay un rango que la limite.

- **mta_tax:** Impuesto MTA de USD 0.50 que se activa automáticamente según la tarifa del taxímetro en uso. **Tipo:** cuantitativa **discreta** porque este impuesto es fijo: solo 0.50 o, si no se aplica, 0.
- **tip_amount:** Monto de la propina. Este campo se completa automáticamente para las propinas pagadas con tarjeta de crédito. Las propinas en efectivo no están incluidas. **Tipo:** cuantitativa **continua** ya que no hay un rango que pueda limitarla y porque representa dinero y admite decimales entre dos valores.
- **tolls_amount:** Monto total de todos los peajes pagados durante el viaje. **Tipo:** cuantitativa **continua** porque, al representar el monto total de peajes, puede contener números decimales y no se encuentra dentro de un rango establecido.
- **improvement_surcharge:** Recarga de mejora de USD 0.30 aplicado a los viajes al momento de iniciar el viaje (flag drop). **Tipo:** cuantitativa **discreta**, ya que se posee dos alternativas: o son 0.30 o 0.
- **total_amount:** El monto total cobrado a los pasajeros. No incluye las propinas en efectivo. **Tipo:** cuantitativa **continua** porque no tiene un rango establecido y no es contable, ya que el total puede tener decimales.
- **congestion_surcharge:** Monto total recaudado en el viaje por el recargo por congestión de NYS (Estado de Nueva York). **Tipo:** cuantitativa **discreta**, ya que, por cómo fue establecido el impuesto, se cobra 0.50 a los viajes grupales o 2.5 en cada viaje con taxi amarillo. Y si no se aplica es 0.
- **Airport_fee:** USD 1.25 sólo por la recogida en los aeropuertos de LaGuardia y John F. Kennedy. **Tipo:** cuantitativa **discreta**, pues toma el valor de 1.25 cuando se recoge en esos aeropuertos o sino vale 0.

Cualitativas

Nuevamente optamos por utilizar la definición propuesta por la cátedra para establecer qué variables correspondían a esta categoría: “...se distinguen por ser diferentes, no se puede establecer un ordenamiento entre ellas...”

- **VendorID:** Un código que indica el proveedor de TPEP (Taxi and Limousine Commission's Taxi Passenger Enhancement Program) que proporcionó el registro. 1 = *Creative Mobile Technologies, LLC*; 2 = *VeriFone*. **Tipo:** cualitativa **numérica nominal** ya que cada número representa una categoría sin orden ni jerarquía específica.
- **RatecodeID:** El código final de tarifa en vigor al final del viaje. 1 = *Tarifa estándar*; 2 = *JFK*; 3 = *Newark*; 4 = *Nassau o Westchester*; 5 = *Tarifa negociada*; 6 = *Viaje en grupo*. **Tipo:** cualitativa **numérica nominal** ya que cada número representa una categoría sin orden ni jerarquía específica.

- **store_and_fwd_flag**: Un indicador que señala si el registro del viaje fue guardado en la memoria del vehículo antes de ser enviado al proveedor, también conocido como 'almacenar y reenviar', porque el vehículo no tenía conexión al servidor. *Y = viaje almacenado y reenviado; N = no es un viaje almacenado y reenviado*. **Tipo**: cualitativa **texto nominal**, ya que “Y” o “N” no tienen jerarquía ni orden entre sí.
- **PULocationID**: Zona de taxi de la TLC (Comisión de Taxis y Limusinas) en la que se activó el taxímetro. **Tipo**: cualitativa **numérica nominal** ya que los números identifican zonas específicas, pero no implican cantidad ni orden.
- **DOLocationID**: Zona de taxi de la TLC en la que se desactivó el taxímetro. **Tipo**: cualitativa **numérica nominal** ya que los números identifican zonas específicas, pero no implican cantidad ni orden.
- **payment_type**: Un código numérico que indica cómo el pasajero pagó el viaje. *1 = Tarjeta de crédito; 2 = Efectivo; 3 = Sin cargo; 4 = Disputa; 5 = Desconocido; 6 = Viaje*. **Tipo**: cualitativa **numérica nominal** ya que cada número representa una categoría sin orden ni jerarquía específica.

Los features más destacables para nosotros fueron **total_amount**, **payment_type**, **passenger_count**, **trip_distance** y **tip_amount**. Consideramos que estas variables brindan una gran cantidad de información sobre cada viaje, ya que permiten conocer lo que costó y cómo se pagó, cuántas personas viajaron (y consecuentemente pagaron), la distancia recorrida y la propina dejada.

Otras variables que nos interesaron fueron **tpep_pickup_datetime** y **tpep_dropoff_datetime**, ya que permiten conocer la duración de los viajes y **PULocationID** y **DOLocationID** porque permiten conocer el lugar donde se inició o terminó un viaje, respectivamente. Estas últimas cuatro que mencionamos (por más interesantes que nos hayan resultado) no son las principales, y solemos utilizarlas en relación con otras variables (más adelante se detallarán).

Supuestos e hipótesis

- A mayor cantidad de pasajeros, más costoso el viaje.
- Los viajes más largos se hacen en hora pico (6:00 a 8:00 AM/PM), viéndose representado como la personas desplazándose hacia su lugar de trabajo.
- En la llegada de estaciones de los barrios más pudientes, la propina es mayor.
- A mayor distancia recorrida, mayor propina.
- A mayor distancia recorrida, mayor cobro en peajes.

Análisis más profundo de los registros

Datos faltantes y eliminación de columnas

Con respecto a la eliminación de columnas, primero analizamos los datos faltantes en el dataset. El análisis concluyó en que cada una de las siguientes variables tiene 1021387 faltantes (10.542%): **RatecodeID**, **Airport_fee**, **congestion_surcharge**, **passenger_count**, **store_and_fwd_flag**.

RatecodeID tiene como valores posibles números del 1 al 6 que indican un tipo de tarifa específico. Para recuperar algunos de esos datos faltantes, analizamos los tipos de tarifas y llegamos a la conclusión de que podíamos recuperar las correspondientes a los números 2, 3 y 4. Esto se debió a que el 2 (Tarifa JFK) se aplica cuando la recogida o la dejada fue en el Aeropuerto JFK, por lo que, si el PULocationID o el DOLocationID eran los correspondientes al aeropuerto, podíamos reemplazar el NaN. Lo mismo sucedió con 3 (Tarifa Newark), que es desde/hacia el Aeropuerto de Newark y con 4 (Tarifa Nassau o Westchester), que queda fuera de Nueva York. Todas las IDs correspondientes a la zona del aeropuerto de JFK, Newark y Nassau o Westchester fueron obtenidas del archivo [taxi zone lookup](#), que es un archivo csv que contiene la ID, el barrio, la zona y la zona de servicio de los viajes en taxi.

En el caso de **Airport_fee**, se aplicaba el impuesto de 1.25 USD si los viajes comenzaron en los aeropuertos de La Guardia y JFK, por lo que procedimos de una forma similar al RatecodeID: nos aseguramos de tener las IDs correspondientes a cada aeropuerto y si el viaje comenzaba en ellos (o sea si la PULocationID concordaba) se asignaba el 1.25, sino 0.

Con la variable **congestion_surcharge** necesitamos investigar un poco más del impuesto, por lo que visitamos la página de [Taxi & Limousine de Nueva York](#) y la página del [Departamento de Impuestos de Nueva York](#). En resumen, el impuesto es un recargo obligatorio por congestión vehicular que se aplica en Manhattan al sur de la calle 96th Street, implementado a partir de enero de 2019. El monto del recargo depende del tipo de vehículo que se utilice para el transporte en o a través de la zona de congestionamiento. El recargo generalmente es \$2.50 por cada viaje cuando el transporte sea en taxi amarillo y \$0.75 por cada viaje compartido. Tras investigar, agrupamos las distintas zonas de [taxi zone lookup](#), filtrando solo las que estaban al sur de la calle 96th Street. Obtuvimos sus IDs y buscamos si había viajes en grupo con NaN por esas zonas. Como no había, no reemplazamos nada y seguimos con los viajes en taxi amarillo por esas zonas, reemplazando los NaN con 2.5.

Respecto a la variable **passenger_count**, observamos que los valores oscilan entre 0 y 9. Para los casos con valores faltantes (NaN), así como para aquellos registros con 0 o 6 pasajeros (considerando que el máximo permitido en los taxis de

Nueva York es de 5 personas) decidimos imputarlos con la moda de la variable. Por otro lado, eliminamos los registros con valores entre 7 y 9, ya que los consideramos atípicos: no sólo es físicamente imposible transportar esa cantidad de personas en un taxi, sino que también es ilegal. Además, la cantidad de registros con estas cifras era muy baja (menos de 40).

Por último, en el caso de **store_and_fwd_flag**, primero queríamos ver su relación con la variable **trip_distance**, ya que pensamos que los viajes más largos tenían más probabilidades de tener **store_and_fwd_flag** nulo, lo cual es coherente con la idea de que los viajes largos podrían verse afectados por falta de conexión durante el recorrido, lo que provoca que el registro se guarde y luego se reenvíe. Pero luego, observamos que los porcentajes de nulos varían de manera significativa según el tramo de distancia, y parece que no existe un patrón claro ni constante para poder sustituir los valores nulos. Por lo que optamos por eliminar los datos nulos de la variable.

Correlación y covarianza

En el análisis de covarianza, no encontramos valores significativamente altos en la mayoría de las variables. Sin embargo, se observó una relación destacada entre las variables **fare_amount** y **total_amount**, lo cual coincide con el análisis de correlación que se realizará más adelante. Este hallazgo tiene sentido, dado que ambas variables están estrechamente relacionadas, siendo la suma de varios cargos que componen el costo total del viaje.

Por otro lado, no surgieron otros valores de covarianza de interés. En el caso de las variables **PULocation** y **DOLocation**, la covarianza no aportó información relevante, ya que se trata de variables cualitativas que no se pueden analizar en este tipo de contexto de manera directa.

En la **matriz de correlación** encontramos valores altos en:

- **fare_amount y total_amount**: Esta relación es probablemente la más intuitiva, ya que, en general, a mayor tarifa base del viaje, mayor será el monto total que paga el pasajero (incluye impuestos, propinas, peajes, entre otros cargos).
- **tip_amount y total_amount**: También es una relación predecible. A medida que el viaje se vuelve más largo o costoso, es más probable que el conductor reciba una mejor propina.
- **mta_tax y improvement_surcharge**: Esta fue la relación más inusual. Aunque ambas variables son teóricamente discretas y tienden a tomar valores fijos, la

presencia de outliers afectó la distribución, reduciendo ligeramente la fuerza de la relación esperada.

Además, aprovechamos esta etapa del análisis para descartar varios de los supuestos iniciales que habíamos planteado. Más adelante detallaremos los motivos de cada descarte.

Generación de nuevos features

Decidimos, con los datos ya existentes, crear los siguientes features:

Features de Imaginación

- **trip_duration:** Consideramos importante incorporar este feature, ya que es una de las pocas variables del tipo cuantitativo numérico. Resulta útil para conocer la duración de cada viaje, identificar la duración mínima dentro de un grupo de viajes o realizar distintos tipos de análisis relacionados.

Features de discretización

- **month:** Nos permite agrupar los viajes según los tres meses en los que ocurrieron. Esto resulta útil ya que, originalmente, los datasets venían separados por mes, pero los unimos por conveniencia. Esta variable nos da la posibilidad de volver a analizarlos por separado si es necesario.
- **day_of_the_week:** Permite agrupar los viajes por día de la semana, lo cual es útil para analizar patrones como los métodos de pago más utilizados en determinados días.
- **bins_trip_distance:** Agrupa los viajes en rangos fijos de distancia. Esto facilita filtrar rápidamente, por ejemplo, los viajes cortos, y enfocarse en ellos para ciertos análisis.
- **rush_hour:** Es una variable booleana que indica si el viaje ocurrió en horario pico. Es útil para dividir los datos entre viajes en hora pico y los que no, y así poder analizar ambos grupos por separado.

Consideramos generar más features basadas en discretización, especialmente sobre montos de dinero, pero resultó ser una tarea más abstracta. ¿Quién define qué es barato o caro? Con las distancias ocurría algo similar aunque en menor medida

(ya que las distancias son objetivas, mientras que los sueldos y percepciones de costo varían entre personas). Por eso optamos por discretizar distancias en lugar de montos.

No se crearon features de normalización. Consideramos que no era relevante en este caso, ya que la mayoría de las variables cuantitativas comparten una característica común: no toman valores negativos. En este contexto, la normalización pierde utilidad, dado que ya sabemos de antemano que los valores son necesariamente positivos, y su escala no representaba un problema para los análisis que realizamos.

Valores atípicos

Con el objetivo de preservar la mayor cantidad posible de información sin comprometer la calidad de los datos, analizamos cuidadosamente los valores atípicos presentes en distintas variables del conjunto de datos. En cada caso, se decidió entre eliminar o imputar los valores extremos en función de su severidad, frecuencia y grado de razonabilidad dentro del contexto del problema.

1. trip_distance

Para esta variable, se utilizó el método de Z-score para detectar valores atípicos:

- Se eliminaron los valores **extremadamente atípicos**, es decir, aquellos con un Z-score mayor a 4. Primero se analizó con un umbral de decisión igual a 3 pero de este modo quedaban demasiados valores marcados como atípicos a pesar de que eran valores con mucha probabilidad de ocurrir, no salían de lo “lógico”, por lo tanto decidimos incrementar el umbral para ver realmente cuales eran los valores más extremos. Un ejemplo a destacar que nos causó asombro y gracia fue que había un viaje que recorrió 330.397,59 millas, cuando una vuelta al mundo son 24,901 millas. Lo que le agrega un toque especial es que, además, se supone que recorrió esa distancia en el rango de 3 meses que nos fue asignado. Investigando más, descubrimos que el viaje duró 14 minutos (pues el tpep_pickup_datetime y tpep_dropoff_datetime son la misma fecha pero con 14 minutos de diferencia...)

```
fila_max_distancia = data[data['trip_distance'] == data['trip_distance'].max()]
fila_max_distancia
```

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	
9434826	2	2024-09-15 14:18:00	2024-09-15 14:32:00	NaN	330397.590000

- Para los valores **moderadamente atípicos** (Z-score entre 3 y 4), se aplicó una **imputación con la mediana** de la variable, considerando que estos pueden deberse a errores de medición menores o situaciones poco frecuentes, pero no imposibles.

2. fare_amount

Para esta variable se eliminaron los valores mayores a **250**, ya que se consideran extremadamente atípicos para una tarifa de taxi. Los valores restantes que superaban levemente ese umbral fueron **imputados con la mediana**, conservando la tendencia general de la variable.

3. tip_amount

Respecto a esta variable, se eliminaron los valores mayores a **200**, por considerarse poco realistas como propina. Los valores atípicos menos extremos fueron **imputados con la mediana**, manteniendo la distribución general sin perder demasiada información.

4. tolls_amount

Con relación a esta variable, se eliminaron los valores mayores a **150**, considerados anómalos en el contexto de peajes en la ciudad. Los valores moderadamente altos fueron **imputados con la mediana**, utilizando el mismo criterio que en las variables anteriores.

5. passenger_count

Sobre `passenger_count`, se identificó que los valores válidos para la variable `passenger_count` se encuentran entre **1 y 5**, que es el rango permitido para los taxis de Nueva York. Se **imputaron con la moda** aquellos registros que presentaban valores faltantes (NaN), o que tenían valores 0 o 6, los cuales se consideran inconsistentes o inusuales. Los registros con valores entre **7 y 9** fueron **eliminados**, ya que representan situaciones físicamente imposibles o ilegales, y además eran muy poco frecuentes.

6. total_amount

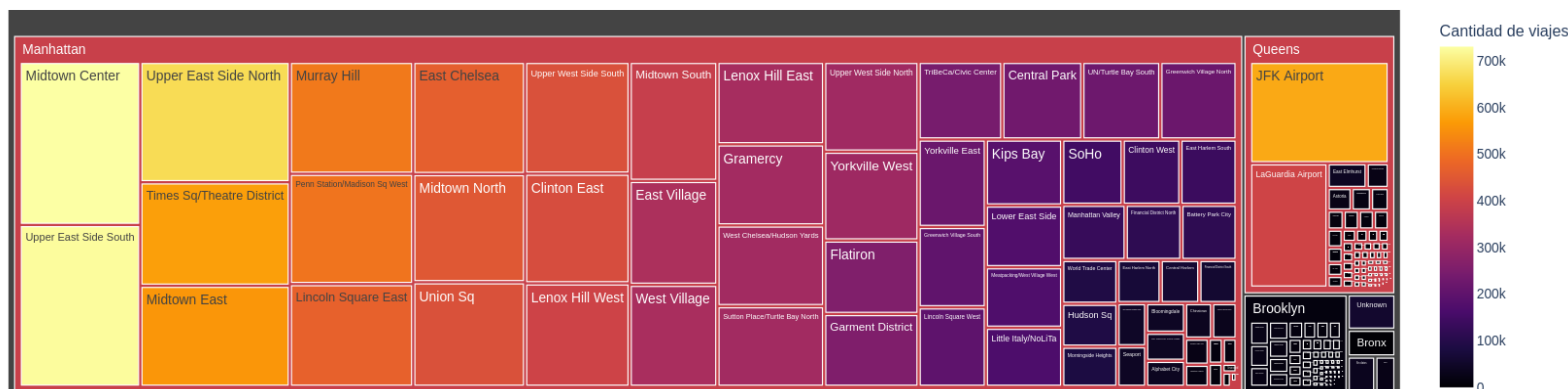
Acerca de total_amount, se eliminaron los registros con un total_amount mayor a **800**, ya que representaban montos excesivos e improbables para un viaje típico. Adicionalmente, se verificó la **coherencia interna** de esta variable. Se comparó el valor de total_amount con la suma de sus componentes (fare_amount, extra, tip_amount y tolls_amount). Si la diferencia era mayor a una **tolerancia de 1 unidad**, y el valor no era extremo, se consideró **incoherente** y se **imputó con la media** de la variable.

Preguntas de investigación

En base a todo el análisis previo y considerando el Dataset, planteamos una serie de preguntas.

1. ¿Qué estaciones de taxi son las más frecuentadas? ¿Cuántos viajes se realizan en las zonas más céntricas de la ciudad?

Estaciones de Taxi mas frecuentadas en NYC



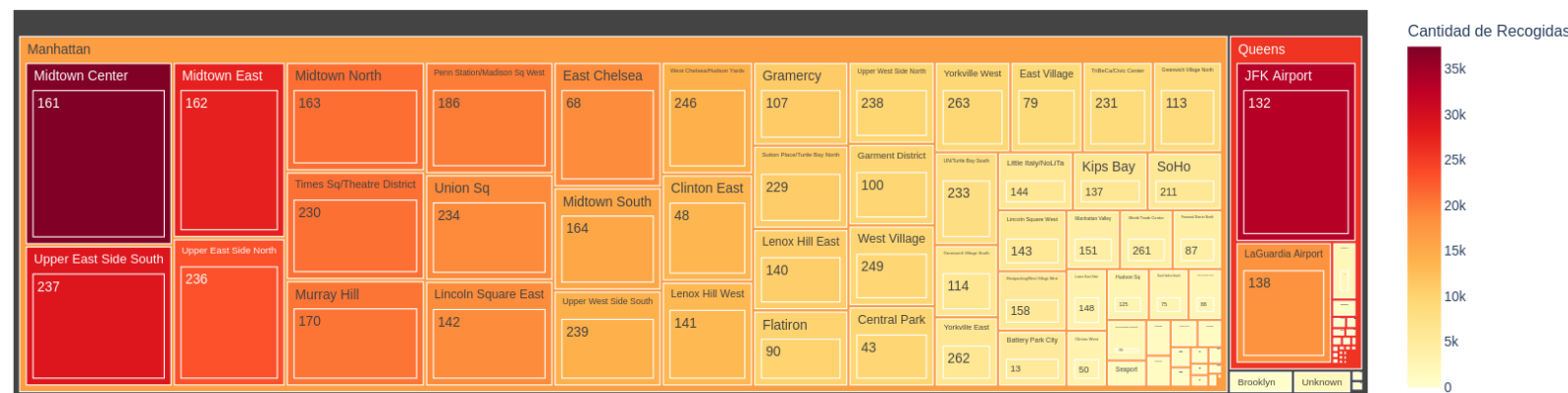
El gráfico tipo *treemap* muestra claramente que las estaciones de taxi más utilizadas se encuentran en **Manhattan**, particularmente en zonas como **Midtown Center**, **Upper East Side North**, **Murray Hill**, y **Times Sq/Theatre District**. Estas áreas están entre las más céntricas, comerciales y densamente pobladas de la ciudad.

Además, se observa que los aeropuertos, especialmente **JFK Airport** y **LaGuardia Airport** en Queens, también concentran una alta cantidad de viajes, lo que es esperable dada la necesidad frecuente de transporte desde y hacia estos puntos.

Podemos concluir que **la mayoría de los viajes en taxi se concentran en las zonas más céntricas y activas de la ciudad**, como Manhattan, donde hay una mayor demanda por motivos laborales, turísticos y de conectividad urbana.

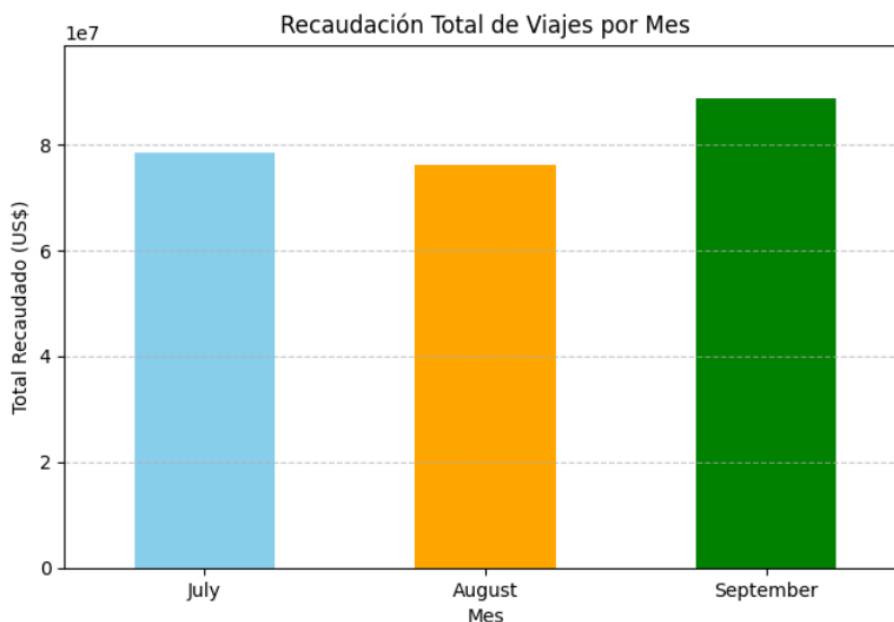
2. ¿En qué estaciones se suben más personas durante la hora pico?

Zonas con más recogidas en hora pico (18:00)



Durante la hora pico de las 18:00, las zonas que registran más recogidas de pasajeros en la ciudad de Nueva York son principalmente las áreas **más céntricas de Manhattan**. Estas zonas se caracterizan por su alta densidad residencial y comercial, lo cual genera una mayor demanda de taxis al finalizar la jornada laboral.

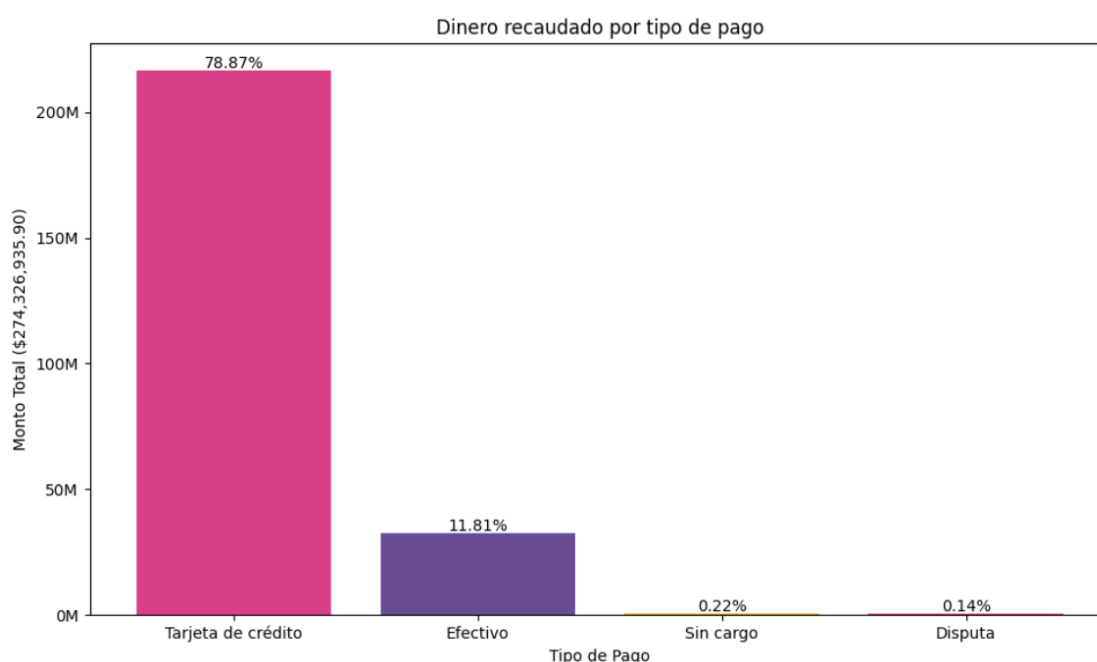
3. ¿Hay algún mes en particular que tenga mayor recaudación?



Según el gráfico, el mes con mayor recaudación es **septiembre**, con un total superior a los 90 millones de dólares (US\$), seguido por **julio**, con una recaudación cercana a los 80 millones y luego por **agosto**, con una leve disminución respecto a julio, quedando como el mes con menor recaudación entre los tres.

Esto sugiere un aumento significativo en la actividad o en las tarifas durante septiembre, lo que podría estar relacionado con el regreso de la actividad laboral o escolar después del verano.

4. ¿Qué método de pago eligen más los pasajeros? ¿Cuánto recaudó cada método de pago?



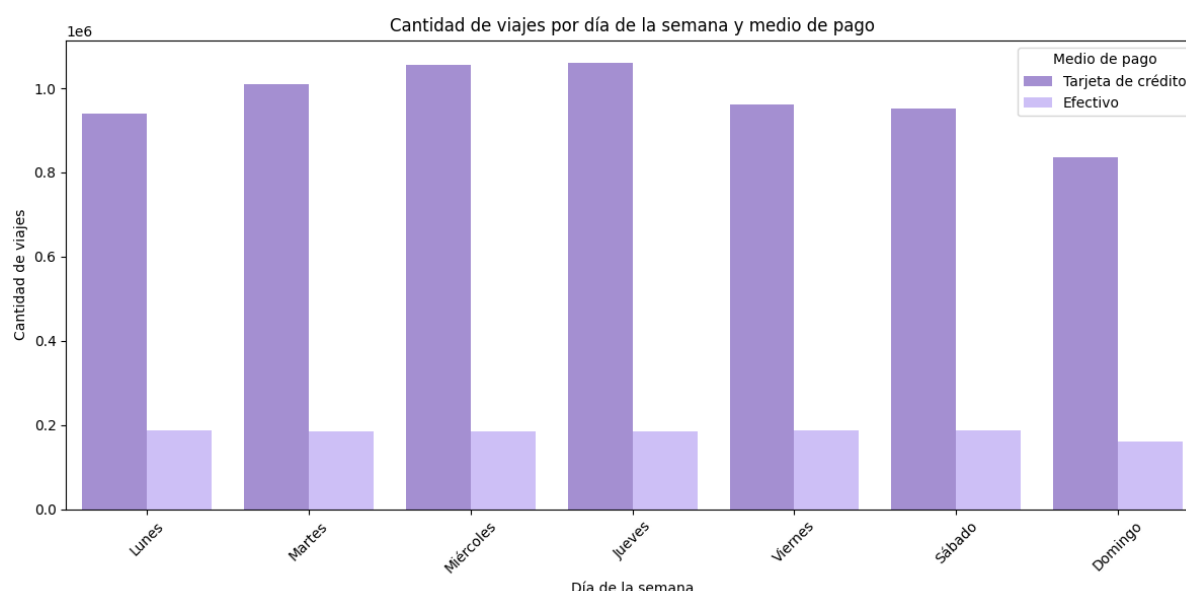
El método de pago más utilizado por los pasajeros es **la tarjeta de crédito**, representando el **78.87%** del total recaudado. Esto indica una clara preferencia por medios electrónicos a la hora de abonar los viajes, lo que puede deberse a factores como comodidad, seguridad, o políticas de incentivo por parte del servicio. La baja participación del pago en efectivo sugiere una tendencia hacia la digitalización de las transacciones en este ámbito.

Además, al incorporar la nueva feature (`day_of_the_week`) que indica el día de la semana, fue posible graficar esta información y analizarla en relación con la cantidad de viajes y el método de pago utilizado. Este gráfico nos permitió responder dos preguntas clave:

5. ¿Qué estaciones de taxi se llevan la mayor cantidad del total de la propina?

Esta pregunta se relaciona con uno de los supuestos que planteamos, por lo que profundizaremos en ella más adelante con su gráfico correspondiente.

6. ¿Cuál es la variación de elección de métodos de pago en base al día de la semana? ¿Hay diferencias significativas entre días laborables y fines de semana en el uso de métodos de pago?



A partir del análisis visual, observamos que los jueves son el día con mayor cantidad de viajes, mientras que los domingos presentan una disminución notable. Además, no se evidencian patrones distintos en la elección de métodos de pago a lo largo de la semana, por lo general predomina el medio de pago tarjeta de crédito por sobre el efectivo.

Supuestos

– Supuestos Descartados

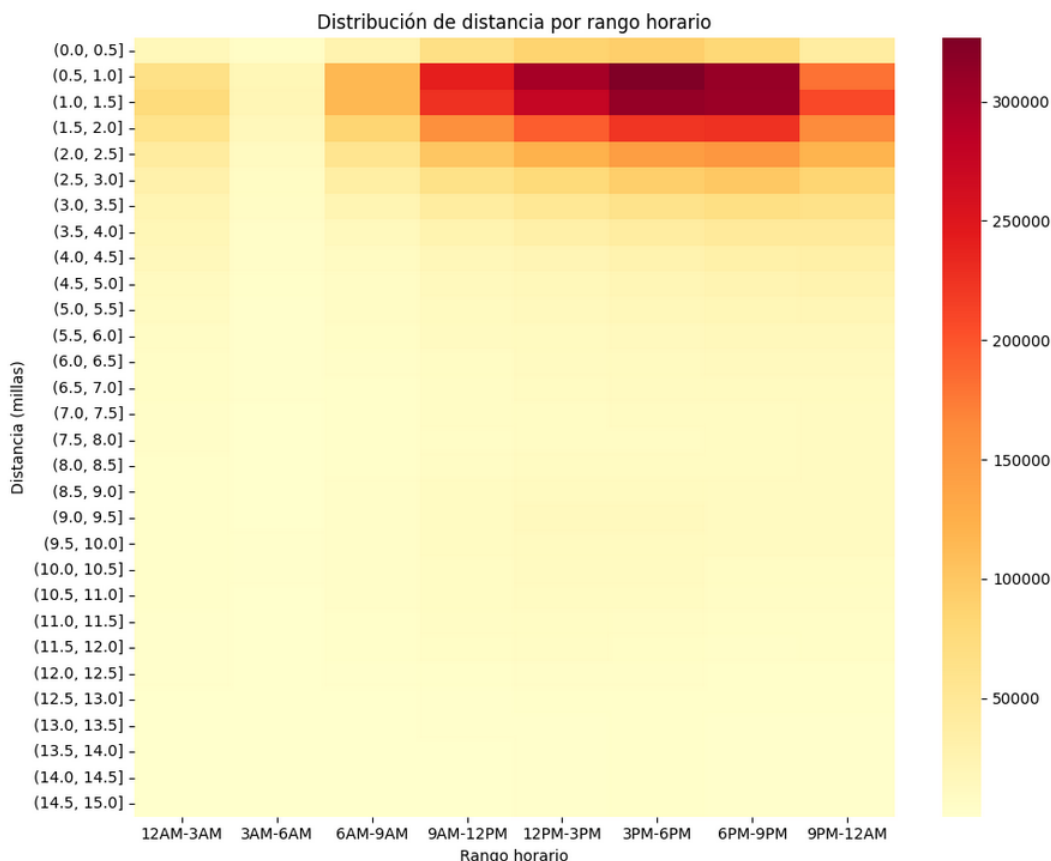
Durante el análisis inicial, planteamos algunas hipótesis intuitivas sobre la relación entre ciertas variables. Sin embargo, al evaluar la correlación entre ellas, observamos que no existía una relación lineal significativa. Dado que la correlación es una condición necesaria para investigar posibles relaciones de causalidad, decidimos descartar estos supuestos:

- **"A mayor cantidad de pasajeros, mayor costo del viaje"**: La correlación entre `passenger_count` y `total_amount` tiende a 0, por lo que no se evidencia una relación clara entre estas variables.
- **"A mayor distancia, mayor cobro en peajes"**: Al analizar la relación entre `trip_distance` y `tolls_amount`, también encontramos una correlación cercana a 0, descartando este supuesto.
- **"A mayor distancia recorrida, mayor propina"**: En este caso, la correlación entre `trip_distance` y `tip_amount` fue nula o muy baja, por lo que tampoco se justifica seguir profundizando esta hipótesis.

En resumen, en los tres casos se concluyó que, al no cumplirse una condición de correlación alta, no había fundamento suficiente para investigar una posible relación causal.

– Supuesto confirmados

Planteamos la hipótesis de que **"los viajes más largos tienden a realizarse durante las horas pico (6:00 a 8:00 AM/PM)"**, como reflejo del desplazamiento de las personas hacia o desde sus lugares de trabajo. Para responder esta pregunta, realizamos un análisis visual a través de un gráfico que mostraba la duración de los viajes segmentada por franja horaria.



Los resultados indicaron que efectivamente, hay un aumento en la cantidad de viajes largos durante las horas pico, especialmente en el rango de la tarde. Sin embargo, la diferencia no fue abismal: en promedio, la cantidad de viajes largos durante esas franjas horarias fue aproximadamente tres veces mayor que en otros momentos del día.

Dado esto, podríamos atribuir el fenómeno más a una cuestión de demanda general de taxis en esas horas específicas que a una relación directa con la longitud del viaje. Aun así, y considerando la evidencia observada, confirmamos este supuesto. **¿Las propinas son mayores al llegar a estaciones ubicadas en barrios más pudientes?**

Esta hipótesis apuntaba a identificar si existe una relación entre el nivel socioeconómico de las zonas de destino y el monto de propina entregado al conductor. Para evaluarla, realizamos un gráfico tipo *treemap* en el que representamos la suma total de propinas recibidas, agrupadas por estaciones de llegada.

El análisis confirmó nuestra suposición: **las estaciones ubicadas en barrios más pudientes concentran un mayor monto acumulado en propinas**. Esto sugiere que los pasajeros que viajan hacia esas zonas, en promedio, tienden a dejar mejores propinas, lo cual podría estar relacionado con su poder adquisitivo o con prácticas culturales asociadas al uso del servicio.

Mora Lema	Análisis e imputación de datos faltantes Análisis de outliers Armado del Reporte Armado de gráficos	8
Lara Eliana lenco	Análisis inicial de los datos faltantes Creación de nuevos features Armado de gráficos	8

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

Ejercicio 2: *Modelos de Clasificación Binaria*

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencko@fi.uba.ar
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar

Fecha de entrega: 01/05/25

Ejercicio 2: Modelos de Clasificación Binaria

Breve descripción y análisis del dataset

Locaciones asignadas: Queensland / Victoria / Australia Meridional / Australia Occidental

El Dataset contiene información sobre distintas estaciones meteorológicas de Australia. El objetivo es predecir si lloverá o no al día siguiente, volcándolo en la variable RainTomorrow. La cantidad de registros inicial del Dataset (sin filtrar por nuestras locaciones asignadas) fue de 145.460 filas y 23 columnas. Tras la filtración por locations, pasó a ser de 78.014 filas.

Luego del primer análisis, definimos si cada variable era cuantitativa o cualitativa, nominal u ordinal y continua o discreta, respectivamente. A las variables cuantitativas les calculamos sus medidas de resumen y para las cualitativas obtuvimos sus frecuencias. que mostramos mediante tablas y gráficos.

La categorización final fue:

Tipo de variable	Cuantitativas	Tipo de variable	Cualitativas
<u>Continua</u>	MinTemp, MaxTemp, Rainfall, Evaporation, Sunshine, WindGustSpeed, WindSpeed9am, WindSpeed3pm, Pressure9am, Pressure3pm, Temp9am, Temp3pm	<u>Nominal</u>	Date, Location, WindGustDir, WindDir9am, WindDir3pm, RainToday, RainTomorrow
<u>Discreta</u>	Humidity9am, Humidity3pm, Cloud9am, Cloud3pm	<u>Ordinal</u>	—

En cuanto a los datos faltantes, primero analizamos el porcentaje de datos faltantes por columna. Creamos un gráfico y descubrimos que de 23 columnas, 21 de ellas tenían valores faltantes. Antes de imputar los valores faltantes, nos cuestionamos si podíamos eliminar alguna columna pero optamos por no hacerlo porque todas parecen aportar información relevante para lograr el objetivo del trabajo. La imputación de valores faltantes fue realizada con la mediana para las variables cuantitativas y por moda para las cualitativas. De esta forma suprimimos todos los valores NaN en el dataset.

Para visualizar correctamente los valores atípicos, hicimos gráficos boxplot por cada variable cuantitativa y luego creamos una tabla donde mostraba la cantidad de outliers por variable basándose en el rango intercuartílico. Vimos los valores extremos tanto bajos como altos. Todo esto fue para nuestro propio entendimiento del Dataset y sus características, pues para ser más precisos optamos por utilizar IsolationForest para la efectiva filtración de los outliers. Tras la filtración, quedaron 74.113 filas (3901 registros menos).

Entrenamiento y predicción

Un conflicto con el que nos encontramos al momento de ver el rendimiento de los distintos modelos fue que las métricas daban valores muy bajos de Recall. En general, todos los modelos no predecían correctamente RainTomorrow, y eran demasiado buenos prediciendo NotRainTomorrow. Tras mucho indagar, descubrimos que el dataset estaba desbalanceado: un 80% del mismo era de la clase NotRainTomorrow, mientras que un 20% era de la clase RainTomorrow.

```
Visualizándolo con porcentajes:  
RainTomorrow  
0    80.059032  
1    19.940968  
Name: proportion, dtype: float64
```

Para solucionar esto, debatimos si aplicar RandomOverSampling o RandomUnderSampling. El OverSampling para balancear duplica ejemplos existentes de la clase minoritaria. Mientras que el UnderSampling reduce la cantidad de instancias de la clase mayoritaria, eliminando datos.

Optamos por aplicar OverSampling para preservar la información, dado que el tamaño de nuestro dataset era reducido y no queríamos perder información. Evaluamos su conveniencia de manera específica para cada modelo.

Modelo 1: Árbol de decisión

Con el objetivo de construir un Árbol de Decisión, utilizamos la métrica RandomSearchCV y luego hicimos **K-fold Cross Validation** con **5 folds**. Los hiperparámetros que optimizamos fueron:

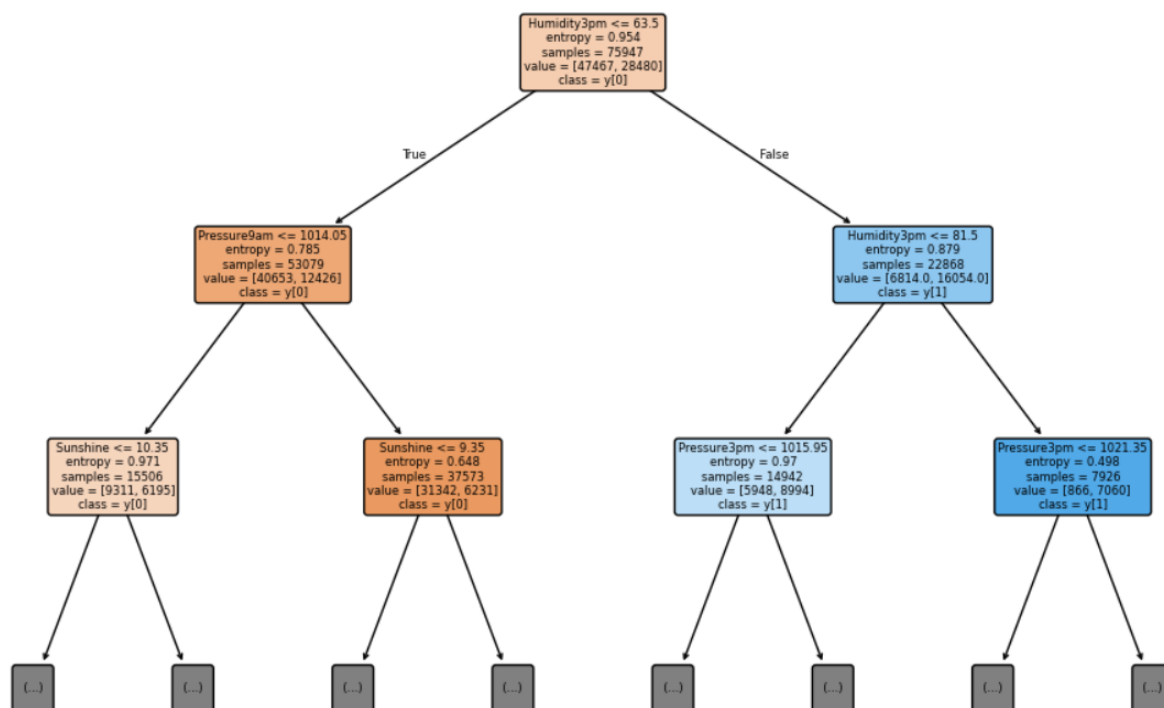
- criterion: criterio de medida para las divisiones ("entropy").
- max_depth: controla la profundidad máxima que puede tener el árbol (randint(4, 10)).

- `min_samples_leaf`: indica la cantidad mínima de muestras que debe tener una hoja de un árbol (`randint(80, 150)`).
- `min_samples_split`: define el número mínimo de muestras necesarias para dividir un nodo (`randint(20, 80)`).
- `ccp_alpha`: controla la poda del árbol y elegimos `uniform(0.000005, 0.00005)` que toma un valor aleatorio dentro de ese rango.

Respecto al OverSampling, lo aplicamos con una `sampling_strategy` de 0.6. El dataset quedó de la siguiente manera:

```
Distribución tras oversampling en train:
RainTomorrow
0    62.500165
1    37.499835
Name: proportion, dtype: float64
```

Dado que el árbol es muy extenso como para mostrarlo completo en el informe, mostraremos hasta el segundo nivel:



Cada nodo interno representa una decisión basada en una característica (feature). Cada rama representa el resultado de esa decisión (True o False). Y cada hoja representa una predicción final.

El nodo raíz es la primera decisión del árbol y en este caso es *Humidity3pm* ≤ 63.5 (si la humedad a las 15:00 hs es menor o igual a 63.5 se toma la rama izquierda, sino la derecha). Este nodo contiene 74.947 muestras y la clase mayoritaria es **NotRainTomorrow**.

En la rama izquierda a *Humidity3pm* ≤ 63.5 , se evalúa si *Pressure9am* ≤ 1014.05 . Esto quiere decir que evalúa si la presión a las 9:00 hs es menor a 1014.05. Tiene 53.079 muestras y su mayoría son **NotRainTomorrow**. La Subrama Izquierda ve si se cumple *Sunshine* ≤ 10.35 que representa la cantidad de horas de sol con 15.506 muestras y en su mayoría **NotRainTomorrow**. Por otro lado, la Subrama Derecha evalúa *Sunshine* ≤ 9.35 y su cantidad de muestras es 37.573 y en su mayoría son **NotRainTomorrow**.

La conclusión a la que llegamos respecto a la rama izquierda y sus subramas es que si hay baja humedad por la tarde (15:00 hs) y si la presión por la mañana (9:00 hs) es menor o igual a 1014.05, se considera la cantidad de horas de sol. En la subrama izquierda, con menos de 10.35 horas de sol, y en la subrama derecha, con menos de 9.35 horas, ambas condiciones se asocian a que no haya lluvia al día siguiente.

Esto significa que, incluso cuando hay menos sol, la baja humedad es el factor determinante que mantiene baja la probabilidad de lluvia.

A su vez en la rama derecha se evalúa si *Humidity3pm* ≤ 81.5 . En este caso tiene 22.868 muestras y en su mayoría **RainTomorrow**. En la Subrama Izquierda se observa que se evalúa *Pressure3pm* ≤ 1015.95 con 14.942 muestras y es **RainTomorrow**. En cambio, la Subrama Derecha observa *Pressure3pm* ≤ 1021.35 . Tiene 7.962 muestras y son **RainTomorrow**.

Observando la rama derecha y sus subramas concluimos que si la humedad a las 15:00hs es menor o igual a 81.5 y si la presión a esa misma hora es menor o igual a 1015.95 es probable que llueva. Esto nos sugiere que una presión relativamente baja junto con alta humedad favorece condiciones de lluvia.

Por otro lado, si la humedad es mayor a 81.5 en la tarde y si la presión es menor o igual a 1021.35, se observa que igual es probable que llueva. Esto indica que incluso con una presión más alta, la presencia de alta humedad es suficientemente significativa para sostener la predicción de lluvia y presión a la tarde fomenta eso.

Modelo 2: Random Forest

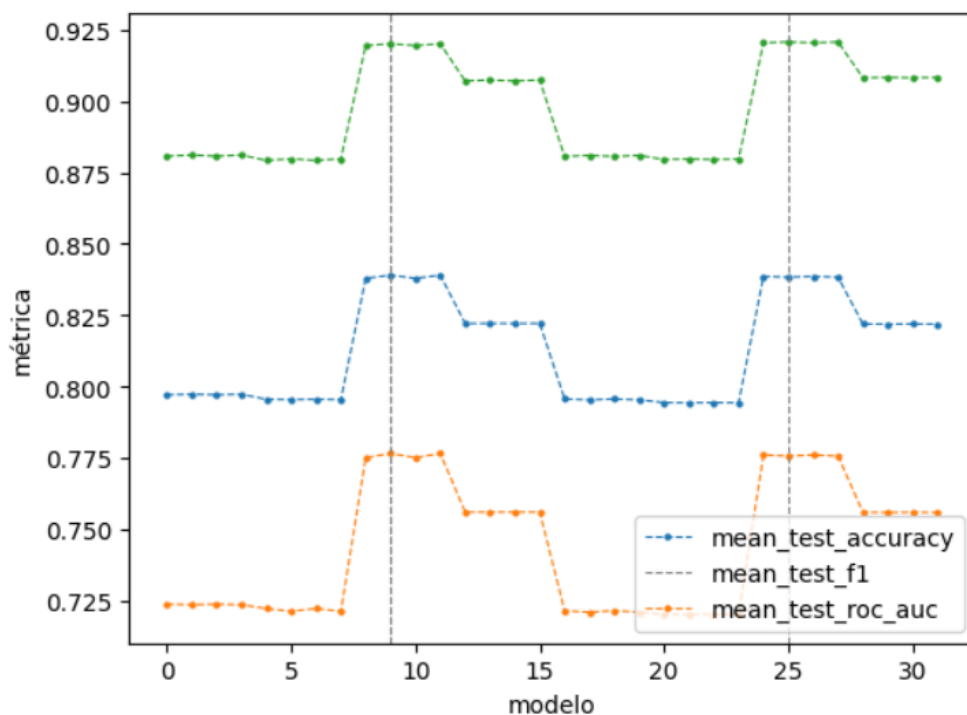
Para crear el Modelo de Random Forest realizamos una **búsqueda de hiperparámetros** utilizando GridSearchCV. Además, utilizamos **K-Fold Cross Validation** con **5 folds** para evaluar el rendimiento de cada combinación de hiperparámetros. Los hiperparámetros optimizados fueron:

- **criterion**: criterio de medida para las divisiones ("gini" o "entropy").
- **min_samples_leaf**: cantidad mínima de muestras en una hoja del árbol (10, 20).
- **min_samples_split**: cantidad mínima de muestras para dividir un nodo (10, 20).
- **n_estimators**: cantidad de árboles en el bosque (100, 200).
- **max_depth**: cuán profundo puede crecer cada árbol individual dentro del bosque (8, 116).
- **class_weight**: que ajuste automáticamente el peso de cada clase en función de cuántas muestras tiene cada una. Así las clases menos representadas tienen más peso, así se les presta más atención. Esto lo que suele mejorar el rendimiento (balanced).

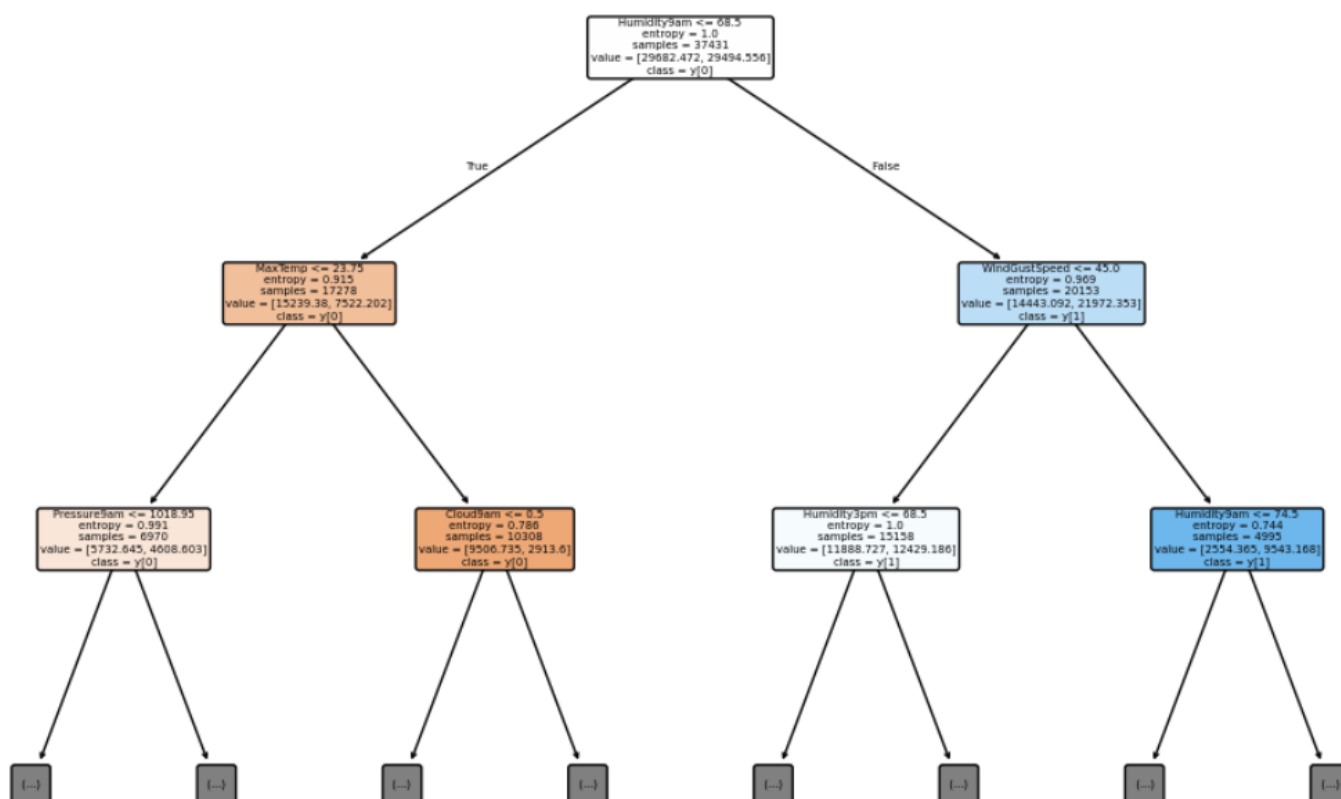
Respecto al OverSampling, lo aplicamos con una `sampling_strategy` de 0.5. El dataset quedó de la siguiente manera:

```
Distribución tras oversampling en train:  
RainTomorrow  
0    66.667135  
1    33.332865  
Name: proportion, dtype: float64
```

Para elegir la mejor combinación de hiperparámetros seleccionamos **F1 Score** como métrica principal ya que consideramos que permite obtener una performance más representativa del comportamiento real del clasificador, al tener en cuenta tanto la precisión como la sensibilidad del modelo frente a la predicción de días con lluvia. Buscábamos un equilibrio entre precisión y recall.



Como la conformación de los árboles generados es muy extensa, mostramos una porción representativa del árbol 19 para explicar las primeras reglas. Lo graficamos con una profundidad máxima de 2 niveles para facilitar su interpretación:



Cada nodo interno representa una decisión basada en una característica (feature). Cada rama representa el resultado de esa decisión (True o False). Y cada hoja representa una predicción final.

El nodo raíz representa la primera decisión del árbol y en nuestro caso utiliza la variable *Humidity9am* ≤ 70.5 (si la humedad a las 9:00 hs es menor o igual a 70.5 se toma la rama izquierda, si es mayor se toma la rama derecha). Este nodo contiene 45.044 muestras, y la clase mayoritaria es **RainTomorrow**.

En la rama izquierda a *Humidity3pm* ≤ 70.5 , tenemos como primer nivel la variable *MaxTemp* ≤ 20.85 . Evalúa si la temperatura máxima es menor o igual a 20.85°C. Esta división tiene 21.738 muestras, y la mayoría son clasificadas como **NotRainTomorrow**. Ahora pasaremos a describir sus subramas: en la Subrama Izquierda se aplica *Pressure9am* ≤ 1019.45 , que evalúa la presión a las 9:00 hs. Su predicción fue **RainTomorrow** con 6.371 muestras. Por otro lado, en la Subrama Derecha se usa *WindGustSpeed* ≤ 49.0 que evalúa la velocidad de la ráfaga de viento más fuerte en km/h. Su predicción también fue **NotRainTomorrow** (con 15.367 muestras).

La conclusión a la que llegamos respecto a la rama izquierda y sus subramas es que si hay baja humedad en la mañana (9:00 hs) tiende a reducir la probabilidad de lluvia al día siguiente. Además si la temperatura máxima del día es más baja tiende a no llover. Pero también sucede que en condiciones de menor temperatura y humedad, una presión baja favorece la lluvia.

Cuando las ráfagas de viento no son fuertes, y la humedad y temperatura son bajas, es menos probable que llueva. Esta rama y sus subramas muestran cómo la baja humedad mañanera, junto con baja temperatura y ráfagas moderadas hace que haya baja probabilidad de lluvias, aunque una presión mañanera más baja puede revertir un poco dicha tendencia.

En la rama derecha (cuando *Humidity3pm* > 70.5) tenemos como primer nivel a *Rainfall* ≤ 0.45 . Este feature considera la cantidad de lluvia caída (en mm). y agrupa 23.306 muestras. La mayoría de estas se clasifican como **RainTomorrow**. En cuanto a sus subramas: en la izquierda tenemos *WindGustSpeed* ≤ 40.0 con una predicción dominante de **NotRainTomorrow** (habiendo evaluado 13.149 muestras). En el caso de la subrama derecha tenemos *Pressure9am* ≤ 1016.05 , que evalúa la presión a las 9:00 hs y su predicción dominante fue **RainTomorrow** (con un total de 10.157 muestras).

Tras haber observado ambas subramas de la rama derecha, concluimos que cuando hay alta humedad en la mañana y si hay algo de lluvia en el mismo día, es probable que al día siguiente llueva también.

En la subrama izquierda, si hay una baja intensidad del viento parece que puede no llover al día siguiente. En cambio, la subrama derecha evalúa la presión mañanera y muestra que, bajo condiciones húmedas y con presión relativamente baja, se incrementa la probabilidad de lluvias.

Modelo 3: Voting Classifier

Para el modelo a elección, optamos por utilizar un ensamble híbrido, siendo este un **Ensamble por Votación (Voting Classifier)**. El ensamble está compuesto por los siguientes modelos: Regresión Logística, Random Forest y K-Nearest Neighbors (K-NN).

Se optimizaron los siguientes hiperparámetros de cada uno de los clasificadores del ensamble:

- **Regresión Logística (lr)**
 - C: parámetro de regularización (valores continuos entre 0.01 y 1)
 - max_iter: cantidad máxima de iteraciones (valores random 100 y 300)
- **Random Forest (rnd)**
 - n_estimators: cantidad de árboles (valores random entre 30 y 100)
 - max_depth: profundidad máxima del árbol ([3, 5, 10])
 - rnd__min_samples_leaf: cantidad mínima de muestras que debe tener una hoja (valores random entre 5 y 15)
 - rnd__max_features: cuántas variables considerará aleatoriamente para dividir en cada nodo del árbol. Usa la raíz cuadrada del número total de variables. ("sqrt")
- **K-Nearest Neighbors (knn)**
 - n_neighbors: cantidad de vecinos a considerar (valores entre 5 y 10)
 - weights: tipo de ponderación de los vecinos, con uniform todos tienen el mismo peso. (uniform)

Utilizamos la validación cruzada (**K-Cross Validation**) con **4 folds** dentro del procedimiento de búsqueda de hiperparámetros. Esto implica que, para cada combinación de hiperparámetros, el conjunto de entrenamiento fue dividido en 4 partes, utilizando 3 para entrenar y 1 para validar, rotando en cada iteración.

Respecto al OverSampling, lo aplicamos con una sampling_strategy de 0.5. El dataset quedó de la siguiente manera:

```
Distribución tras oversampling en train:
RainTomorrow
0    66.667135
1    33.332865
Name: proportion, dtype: float64
```

Para llevar a cabo esta búsqueda, se utilizó **RandomizedSearchCV**, el cual realiza una búsqueda aleatoria sobre un número fijo de combinaciones de hiperparámetros (en este caso, n_iter = 30)

La métrica utilizada para evaluar cada combinación de hiperparámetros durante el proceso de búsqueda fue el **F1 Score**. Esto significa que la mejor combinación fue aquella que maximiza el equilibrio entre precisión y recall en las predicciones evaluadas mediante validación cruzada.

Cuadro de resultados

Medidas de rendimiento del conjunto TEST:

<u>Modelo</u>	<u>F1-Test</u>	<u>Precision Test</u>	<u>Recall Test</u>	<u>Accuracy Test</u>
Árbol de Decisión	0.6105	0.5599	0.6711	0.8273
RandomForest	0.6404	0.6569	0.6246	0.8585
Personalizado (Voting)	0.6364	0.6422	0.6306	0.8547

Parámetros elegidos del Árbol de Decisión:

```
{'ccp_alpha': np.float64(2.4146343737689492e-05), 'criterion': 'entropy', 'max_depth': 9, 'min_samples_leaf': 82, 'min_samples_split': 20}
```

Parámetros elegidos del RandomForest:

```
{'class_weight': 'balanced', 'criterion': 'gini', 'max_depth': 16, 'min_samples_leaf': 10, 'min_samples_split': 10, 'n_estimators': 200}
```

Parámetros elegidos del personalizado:

```
{'knn__n_neighbors': 7, 'knn__weights': 'uniform', 'lr__C': np.float64(0.6218528947223795), 'lr__max_iter': 269, 'rnd__max_depth': 10, 'rnd__max_features': 'sqrt', 'rnd__min_samples_leaf': 7, 'rnd__n_estimators': 84}
```

Significado de las distintas medidas de rendimiento:

- **F1-Test:** es la media entre la *precisión* y el *recall*, que busca equilibrar ambos aspectos. Se calcula como $2 * ((\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}))$.
- **Precision Test:** evalúa la proporción de verdaderos positivos sobre el total de predicciones positivas realizadas. Es $TP / (TP + FP)$
- **Recall Test:** mide la capacidad del modelo para identificar correctamente todos los casos positivos. Es $TP / (TP + FN)$.
- **Accuracy Test:** mide la proporción de predicciones correctas sobre el total de predicciones realizadas. Es $(TP + TN) / \text{total de observaciones}$.

Analicemos la performance de todos los modelos respecto al set de entrenamiento.

En el caso del **Árbol de Decisión**, el set de entrenamiento tuvo la siguiente performance:

```
--- Entrenamiento ---  
Accuracy : 0.8087  
Precision: 0.7673  
Recall    : 0.7032  
F1 Score  : 0.7338
```

Vimos que tuvo un rendimiento moderado en el conjunto de test, destacando por un buen recall (0.67), lo que indica que es capaz de detectar bastante bien los días con lluvia. A pesar de eso, su precisión en test fue baja (0.56) comparado con la del entrenamiento (0.76), lo que sugiere varios falsos positivos. Esto implica que el modelo tiende a predecir lluvia incluso cuando no la hay, lo cual puede ser útil si es más importante la lluvia, pero genera muchas falsas alarmas. A pesar de esto, no vimos un gran sobreajuste, ya que el rendimiento en entrenamiento no es excesivamente superior al de test.

Respecto al **RandomForest**, el set de entrenamiento tuvo la siguiente performance:

```
--- Entrenamiento ---  
Accuracy : 0.8995  
Precision: 0.8853  
Recall    : 0.8026  
F1 Score  : 0.8419
```

Consideramos que las métricas de entrenamiento son altas, lo que significa que el modelo capta bien los patrones del entrenamiento. En general, es el que mejor rendimiento obtuvo en test y entrenamiento, con métricas altas en precisión, recall y F1 Score.

Creemos que puede haber un cierto grado de sobreajuste porque hay mayor diferencia entre test y train, pero no es un overfitting demasiado elevado.

Y, finalmente, el set de entrenamiento del **modelo a elección (VotingClassifier)** tuvo la siguiente performance:

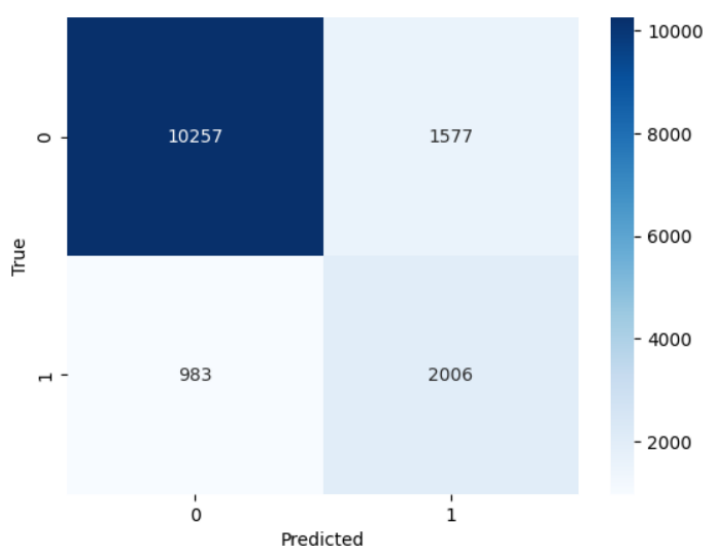
```
--- Entrenamiento ---  
Accuracy : 0.8718  
Precision: 0.6820  
Recall    : 0.6690  
F1 Score  : 0.6754
```

Observamos que el modelo es el más estable entre entrenamiento y test. Aunque sus métricas no fueron las más altas (superadas por el RandomForest), mantuvo un rendimiento constante con un F1 Score de 0.64 en test, muy cercano al de entrenamiento. Además, tiene equilibrio entre precisión y recall lo que lo hace una opción buena, aunque no se compara al rendimiento del Random Forest.

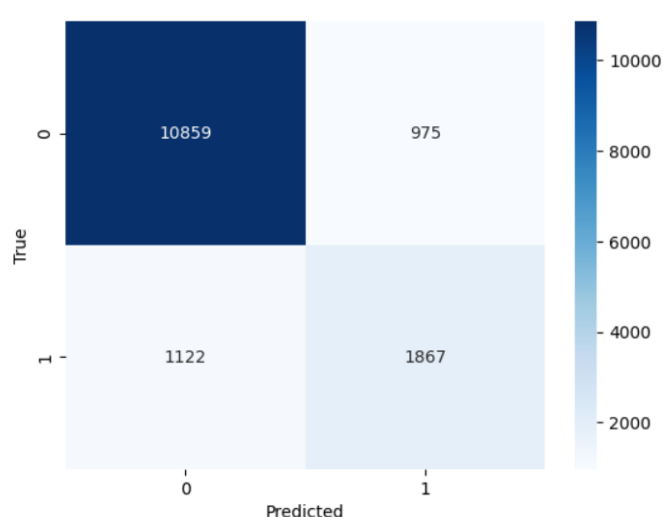
Para definir qué modelo elegiríamos si tuviéramos que predecir si lloverá o no al día siguiente, optamos por observar las matrices de confusión y la tabla de resultados.

Matrices de confusión:

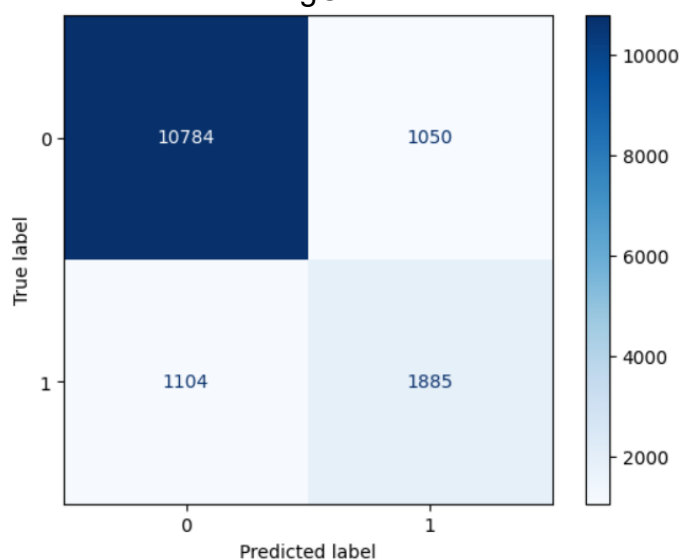
Árbol de Decisión



RandomForest



VotingClassifier



Fue una decisión difícil pero tras debatirlo concluimos que elegiríamos el **RandomForest** ya que es el que tiene mejores métricas a excepción del recall, que difiere por 0.006 con el VotingClassifier. Además, destacamos que el modelo alcanza el mejor F1 Score en el conjunto de prueba, lo que indica un equilibrio sólido entre precisión y recall, por lo que la diferencia con el VotingClassifier es aún menos significativa. Buscamos que estuviera balanceado, para que pudiera identificar correctamente los días en los que lloverá sin generar demasiadas falsas alarmas.

También su rendimiento en entrenamiento fue bueno, lo que muestra que tiene una alta capacidad predictiva. Teniendo en cuenta que aún hay un leve desbalance de clases, predice muy bien.

Analizando las tres matrices de confusión, la del Random Forest muestra que es el modelo que logra el mayor número de verdaderos negativos (10.859), lo que indica que es el más preciso al predecir que no lloverá. Solo comete 975 falsos positivos, que es el valor más bajo de los tres.

A pesar de que tiene un poco más de falsos negativos (1.122) que el Árbol de Decisión (983) o el VotingClassifier (1.104), mantiene un equilibrio razonable entre ambos tipos de error. Esto es clave porque, en un dataset desbalanceado como el nuestro, muchos modelos tienden a sobrepredecir la clase mayoritaria. Sin embargo, el Random Forest muestra ser capaz de identificar correctamente la clase minoritaria (RainTomorrow) con una tasa aceptable de falsos negativos, mientras que a la vez minimiza los falsos positivos, lo que lo convierte en una opción robusta si buscamos evitar alarmas innecesarias (que prediga lluvia cuando no la hay) sin perder demasiadas predicciones de lluvia reales.

Cuadro de tiempo dedicado

Integrante	Tarea	Prom. Hs Semana
Ramiro Barcala Roca	Implementación del Árbol de Decisión	6
Rafaela Pilar Arias	Implementación del RandomForest Implementación del Personalizado (VotingClassifier) Armado del Reporte	9
Mora Lema	Implementación del RandomForest Implementación del Personalizado (VotingClassifier) Armado del Reporte	9
Lara Eliana Ienco	Análisis inicial de variables	6

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

Ejercicio 3: Regresión

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencho@fi.uba.ar
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar

Fecha de entrega: 01/05/25

Ejercicio 3: Regresión

Breve descripción y análisis del dataset

Ciudad asignada: Río de Janeiro, Brasil.

El dataset contiene datos de alojamiento de la plataforma Airbnb. El objetivo es, mediante diversos modelos, predecir el precio de alquiler de una propiedad en función de los datos publicados en el aviso. La cantidad de registros inicial del Dataset (sin filtrar por la ciudad asignada) era de 39.499 filas y 75 columnas. Tras la filtración por la ciudad de Río de Janeiro, pasó a tener 24.442 filas y la misma cantidad de columnas.

Luego de un breve análisis teniendo en cuenta la descripción de las variables (disponible en el Colab ya que es muy extensa) concluimos en la siguiente categorización:

Tipo de variable	Cuantitativas	Tipo de variable	Cualitativas
Continua	last_scraped, host_since, host_response_rate, host_acceptance_rate, latitude, longitude, price, minimum_nights_avg_ntm, maximum_nights_avg_ntm, calendar_updated, calendar_last_scraped, first_review, last_review, review_scores_rating, review_scores_accuracy, review_scores_cleanliness, review_scores_checkin, review_scores_communication, review_scores_location, review_scores_value, reviews_per_month	Nominal	listing_url, source, name, description, neighborhood_overview, picture_url, host_url, host_name, host_location, host_is_superhost, host_thumbnail_url, host_picture_url, host_neighbourhood, host_verifications, host_has_profile_pic, host_identity_verified, neighbourhood, neighbourhood_cleansed, neighbourhood_group_cleansed, property_type, room_type, bathrooms_text, amenities, has_availability, license, instant_bookable, host_about
Discreta	id, scrape_id, host_id, host_listings_count, host_total_listings_count, accommodates, bathrooms, bedrooms, beds, minimum_nights, maximum_nights, minimum_minimum_nights, maximum_minimum_nights, minimum_maximum_nights,	Ordinal	host_response_time

	maximum_maximum_nights, availability_30, availability_60, availability_90, availability_365, number_of_reviews, number_of_reviews_ltm, number_of_reviews_l30d, calculated_host_listings_count,calculated_h ost_listings_count_entire_homes, calculated_host_listings_count_private_roo ms, calculated_host_listings_count_shared_roo ms		
--	---	--	--

Una vez categorizadas, hicimos el análisis correspondiente, observando sus frecuencias, los posibles valores que pueden tomar y sus medidas de resumen.

Tras haber finalizado el análisis inicial de Dataset, observamos las correlaciones y la covarianza. La matriz de correlación demostró que no había fuertes correlaciones en el Dataset, ya sea negativas o positivas. Y sobre todo, que no había correlaciones significativas con price, variable sobre la cual realizamos el análisis predictivo. Hicimos un listado de las correlaciones con price de mayor a menor y la correlación más fuerte era con bathrooms, siendo de solo 0.23. En cuanto a la covarianza, la matriz mostró resultados más elevados, con la mayor covarianza siendo entre id y host_id. Pero eso no quiere decir que fueran relevantes para predecir el precio pues ambos son identificadores únicos, por lo cual tiene sentido que siempre sea un valor distinto del otro y que varíen siempre. Además de esas dos variables, observamos otras que sí tenían covarianza alta y que podrían ser útiles para el análisis.

El siguiente paso fue analizar los datos faltantes del Dataset. Realizamos un gráfico sobre el porcentaje de valores faltantes por variable y notamos que había tres que estaban compuestas 100% por valores NaN, por lo que las eliminamos ya que no eran imputables por nada. El resto de variables, al ser tantas (en este momento 72) imputamos las cuantitativas por mediana y las cualitativas por moda. Continuamos observando los valores atípicos: realizamos boxplots de cada variable para poder observar en más detalle y luego medimos la cantidad de outliers utilizando el IQR (valores fuera del rango intercuartil) y usando el Z-Score con umbral de 3.

Como modelo personalizado, optamos por elegir el K-Nearest Neighbors (KNN), modelo muy sensible a los outliers; por este motivo filtramos los valores atípicos y el Dataset acabó teniendo 18.342 filas.

Entrenamiento y predicción

Para entrenar los modelos, decidimos eliminar algunas columnas que consideramos irrelevantes para la predicción de price. Quitamos:

- **Identificadores y URLs** (como id, scrape_id, listing_url, host_id, host_url, picture_url, host_thumbnail_url, host_picture_url) pues son únicos para cada fila y no representan patrones que puedan aprender los modelos. Además, son enlaces y no nos interesan en este caso.
- **Textos largos o redundantes** (como name, description, neighborhood_overview, host_about, host_name, host_location, host_neighbourhood, host_verifications, bathrooms_text) pues tampoco consideramos que nos ayudara a predecir el precio.
- **La variable auxiliar porcentaje_nulos**, que nos fue útil para el análisis exploratorio pero no corresponde incluirla como feature del modelo.

Además, transformamos algunas columnas con el fin de que puedan ser encodeadas y normalizadas. Las **columnas con fechas** (last_scraped, calendar_last_scraped, first_review, last_review, host_since) fueron convertidas a antigüedad en días. Realizamos esto pues el valor absoluto de una fecha no aporta información al modelo pero, en cambio, la antigüedad (por ejemplo, cuántos días pasaron desde la última reseña o desde que el host se unió) puede ser informativa sobre el estado actual del alojamiento o del comportamiento del anfitrión.

La columna **amenities**, que contiene una lista de servicios como texto, fue transformada a una variable numérica representando la cantidad total de amenities disponibles (amenities_count). Esta simplificación permite evitar una expansión excesiva de columnas si se hiciera one-hot encoding y también aportar un valor que se correlaciona intuitivamente con el precio (más servicios, mayor precio esperado).

También se crearon tres nuevas variables a partir de columnas de texto descriptivo. Estas nuevas variables cuantifican la cantidad de caracteres presentes en las descripciones originales, permitiendo conservar información relevante sobre la extensión de los textos sin necesidad de trabajar directamente con ellos. A continuación, se detallan los cambios:

host_about_length representa la cantidad de caracteres en la descripción proporcionada por el anfitrión sobre sí mismo. Captura de manera numérica la presencia y extensión de la información sobre el anfitrión, lo cual podría correlacionar con la confianza o el profesionalismo percibido.

neighborhood_overview_length indica el número de caracteres en la descripción del vecindario hecha por el anfitrión. Resume la cantidad de detalle proporcionado sobre el entorno del alojamiento, potencialmente relacionado con la calidad o el atractivo del anuncio.

description_length mide la cantidad de caracteres en la descripción principal del anuncio. Permite evaluar de forma numérica cuán extensa es la descripción del alojamiento, información que puede ser indicativa del nivel de detalle o marketing del anuncio

Luego de crear estas nuevas variables, las columnas originales (last_scraped, calendar_last_scraped, first_review, last_review, host_since, amenities, host_about, neighborhood_overview y description) fueron eliminadas del conjunto de datos para evitar redundancia y reducir la dimensionalidad.

Posteriormente, realizamos One-Hot Encoding sobre las variables cualitativas: has_availability, instant_bookable, host_has_profile_pic, host_identity_verified, host_is_superhost, neighbourhood_cleansed.

La normalización fue realizada según cada modelo lo necesitara.

Modelo 1: Regresión Lineal Múltiple

En este modelo fue necesario aplicar un proceso de normalización sobre los datos. Una vez normalizados, construimos un regresor lineal siguiendo la metodología vista en clase y procedimos a entrenar el modelo. La performance obtenida fue baja, tanto en el conjunto de entrenamiento como en el de prueba. Para intentar mejorarla, analizamos los coeficientes de la regresión lineal, enfocándonos en los 10 más relevantes. Estos coeficientes reflejan la importancia que el modelo asigna a cada variable: un coeficiente positivo implica que la variable contribuye a aumentar la predicción, mientras que un coeficiente negativo tiende a disminuirla.

Además, realizamos ajustes en el tratamiento de los outliers. Aunque inicialmente los filtramos utilizando un umbral de Z-Score de 4, decidimos hacerlo más estricto y bajarlo a 3. Sin embargo, este cambio no tuvo un impacto positivo en el rendimiento. También intentamos mejorar el modelo revisando el conjunto de datos, eliminando columnas que consideramos poco relevantes y explorando la creación de nuevos features. A pesar de estos esfuerzos, no logramos mejoras significativas.

Finalmente, concluimos que las bajas correlaciones observadas entre las variables disponibles y el precio influyeron negativamente en el desempeño general del modelo.

Modelo 2: XGBoost

En el caso de este modelo, no fue necesario normalizar y solo creamos dummies. Utilizamos K-Fold Cross Validation con 5 folds para mejorar la performance del modelo. También optamos por usar RandomizedSearchCV para hallar los mejores hiperparámetros, ya que es más rápido que GridSearchCV y porque nuestro espacio de búsqueda era grande. Los hiperparámetros que definimos fueron los siguientes:

- `n_estimators`: cantidad de árboles en el bosque (700, 800, 900)
- `max_depth`: controla la profundidad máxima que puede tener el árbol (5),
- `learning_rate`: cada árbol nuevo agrega una corrección y el `learning_rate` define qué tan fuerte es dicha corrección (0.03, 0.04)
- `subsample`: qué porcentaje de los datos ve cada árbol (0.7, 0.8)
- `colsample_bytree`: qué porcentaje de las columnas (features) ve cada árbol (0.7, 0.8)
- `gamma`: divide el árbol según si la mejora es buena (1, 1.5)
- `lambda`: penaliza los pesos de las hojas si son muy grandes (2, 3)
- `alpha`: promueve que algunas hojas tengan peso cero para evitar overfitting (1.0, 1.5)

El desempeño en los conjuntos de entrenamiento y prueba fue mejor con este modelo. Si bien se observó un leve overfitting en el conjunto de entrenamiento, el rendimiento general fue superior al de los modelos anteriores, lo que era esperable dado que se trata de un modelo más potente. Profundizaremos sobre este resultado en la sección de conclusiones.

Modelo 3: K-Nearest Neighbors (K-NN)

Para trabajar con este modelo primero verificamos que no hubiera valores NaN, ya que KNN no los puede manejar. Luego convertimos todas las variables categóricas a numéricas utilizando el One-Hot Encoding. Y, por último, como KNN se basa en distancias (como por ejemplo la euclídea), normalizamos las variables cuantitativas.

Una vez finalizadas las preparaciones iniciales, procedimos a la creación del modelo. Sin embargo, el rendimiento obtenido fue insatisfactorio: se observó un overfitting marcado en el conjunto de entrenamiento y un desempeño extremadamente bajo en el conjunto de prueba, independientemente de los ajustes realizados sobre los hiperparámetros.

Como primera estrategia, implementamos el uso de Pipelining, aunque esto no logró mejorar los resultados. Posteriormente, optamos por la creación de nuevas variables. Si bien esta estrategia no logró corregir el overfitting en el entrenamiento, sí permitió una mejora en el desempeño sobre el conjunto de prueba. La mejora más significativa se reflejó en el valor del R^2 , que pasó de 0,38 a 0,42.

Para la creación de nuevas features tomamos todas las correlaciones del Dataset y vimos cuáles podíamos implementar. Optamos por quedarnos con las siguientes features nuevas:

- **proporcion_listings_completos:** la proporción entre el número de `calculated_host_listings_count_entire_homes` (viviendas completas listadas por el anfitrión) y el número total de posts listados del anfitrión. Indica cuántos de los listados del anfitrión son viviendas completas.
- **proporcion_availability:** la proporción entre la disponibilidad a 30 días y la disponibilidad a 365 días. Indica qué tan disponible está un alquiler en los próximos 30 días en comparación con todo el año.
- **promedio_review_scores:** el promedio de los distintos scores sobre la experiencia de los usuarios (precisión, limpieza, check-in, valor, ubicación y evaluación general). Representa una calificación promedio general del alojamiento basada en diversas dimensiones de la experiencia del huésped.
- **diferencia_review_scores_clean_accuracy:** la diferencia entre las calificaciones de limpieza y precisión de la reseña. Indica si los huéspedes están más enfocados en la limpieza o en la precisión del anuncio.
- **cantidad_bathrooms_bedrooms:** la cantidad de baños por cantidad de dormitorios.

Tras la selección de estas features, medimos las correlaciones de todas las variables con `price`, incluyendo las que habíamos creado. Las correlaciones mejoraron respecto a las que vimos en el análisis inicial gracias a las nuevas features, por lo que nos quedamos con las variables que poseían una correlación mayor o igual a 0.02 y menor a 0 (para incluir a las negativas, que también aportan información).

Luego definimos el KNN, para el cual dividimos los datos usando la transformación logarítmica. Esta toma el valor de la variable y calcula su logaritmo, para reducir la amplitud de las diferencias entre valores grandes y pequeños. Esto lo hicimos porque valores como el precio pueden estar muy sesgados hacia valores pequeños, pero tener unos pocos valores extremadamente grandes y la transformación logarítmica nos ayuda a comprimir estos valores grandes, haciendo que los datos sean más simétricos y fáciles de manejar para el modelo.

En el KNN utilizamos RandomizedSearchCV con 5 folds, realizando una validación cruzada internamente mientras busca los mejores hiperparámetros. Los hiperparámetros que definimos son:

- **n_neighbors**: número de vecinos que el modelo considera para hacer una predicción (3, 5, 7, 9, 11, 13, 15, 17)
- **weights**: cómo se combinan los vecinos para hacer la predicción, cuáles tienen más peso al hacer la ponderación ('uniform', 'distance')
- **metric**: la medida de distancia para saber quiénes son los vecinos más cercanos ('euclidean', 'manhattan', 'chebyshev')

Cuadro de resultados

Medidas de rendimiento del conjunto TEST:

<u>Modelo</u>	<u>MSE</u>	<u>RMSE</u>	<u>R²</u>	<u>MAE</u>
Regresión lineal	663321.02	814.45	0.3467	448.59
XGBoost	441778.32	664.66	0.5649	350.74
KNN	586357.66	765.74	0.4225	388.24

Mejores parámetros encontrados para el XGBoost:

{'subsample': 0.8, 'n_estimators': 900, 'max_depth': 5, 'learning_rate': 0.03, 'lambda': 3, 'gamma': 1, 'colsample_bytree': 0.7, 'alpha': 1.5}

Mejores parámetros encontrados para el KNN:

{'weights': 'distance', 'n_neighbors': 13, 'metric': 'manhattan'}

Significado de las distintas medidas de rendimiento:

- **MSE**: el Error Cuadrático Medio mide el promedio de los errores al cuadrado entre los valores predichos y los valores reales.
- **RMSE**: el Raíz del Error Cuadrático Medio (RMSE) es la raíz cuadrada del MSE. Expresa el error en las mismas unidades que la variable que se está usando.
- **R²**: el Coeficiente de Determinación es la proporción de variabilidad explicada por el modelo. Su valor está entre 0 y 1. Cuando es 0 el modelo se ajusta poco a la variabilidad de los datos. Si es 1 se ajusta mucho a la variabilidad de los datos. Mientras mayor es R², mayor es la fuerza de la/s variable/s regresora/s para predecir la respuesta.
- **MAE**: el Error Absoluto Medio (MAE) mide el promedio de las diferencias absolutas entre los valores predichos y los valores reales.

Analicemos la performance de todos los modelos respecto al set de entrenamiento.

En el caso del modelo de Regresión Lineal Múltiple, el entrenamiento tuvo el siguiente rendimiento:

```
--- Train ---  
MAE: 468.57  
MSE: 738429.01  
RMSE: 859.32  
R2: 0.3710
```

Si lo comparamos con el test, en ambos casos el R^2 es bajo. En test es 0.346. Al ser similares, no nos sugiere que el modelo posea overfitting. Quizás se podría proponer que tiene un poco de underfitting, pero aún así (y comparado al KNN) no consideramos que sea tan grave. También tenemos en cuenta la nula correlación de las variables con price, pues el modelo de Regresión solo tiene en cuenta las relaciones lineales, por lo que su rendimiento malo se debe a eso.

El XGBoost tuvo el siguiente rendimiento en train:

```
--- Train ---  
MAE: 264.05  
MSE: 197073.38  
RMSE: 443.93  
R2: 0.8321
```

En este caso el entrenamiento no tiene tanto overfitting como todos los demás y tiene un MAE menor al de Regresión y mayor al de KNN. Tiene un buen desempeño en el entrenamiento, pudiendo explicar el 83% de la variabilidad de los precios, con un MAE no tan grande. La diferencia con test es lo que nos sugiere un poco de overfitting, ya que el R^2 de test es 0.565, lo cual difiere mucho del train. Aún así, es el que mejor se desempeña de todos los modelos y el MAE del train es de 350, que sigue sin ser excesivo observando el train.

El K-Nearest Neighbors tuvo el siguiente rendimiento en train:

```
--- Train ---  
MAE: 0.03  
MSE: 2.32  
RMSE: 1.52  
R2: 1.0000
```

En el conjunto de entrenamiento se aprecia un overfitting perfecto, sobre todo por el R^2 que es igual a 1. En cambio, en el conjunto test se aprecia un R^2 de 0.42,

que es aceptable considerando la baja correlación inicial. Esto quiere decir que el

modelo puede explicar el 42% de la variabilidad de los precios. Además, el MAE nos dice que el modelo se equivoca en aproximadamente 388 dólares por predicción. Por lo tanto, no es el mejor modelo y lo peor es el train, pero no hallamos una alternativa para disminuir el overfitting del train y subir el rendimiento de test.

Luego de nuestro análisis del dataset en esta ocasión, concluimos que elegiríamos XGBoost para predecir el precio de un alquiler de AirBnB en la ciudad de Río de Janeiro, pues fue el que mejor rendimiento tuvo en test y en train. Es el mejor de los tres en lograr capturar patrones para la predicción del precio y es el que posee un MAE menor y más razonable. Además su RMSE es moderado en comparación a los otros dos modelos, lo que refuerza su elección como el modelo más robusto.

Cuadro de tiempo dedicado

Integrante	Tarea	Prom. Hs Semana
Ramiro Barcala Roca	Implementación del XGBoost	6
Rafaela Pilar Arias	Implementación de Regresión Lineal Análisis inicial de variables Armado del reporte	8
Mora Lema	Análisis inicial de variables Implementación del KNN Armado del reporte	8
Lara Eliana Ienco	Implementación de Regresión Lineal	4

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

Ejercicio 4: Clustering

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencho@fi.uba.ar
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar

Fecha de entrega: 01/05/25

Ejercicio 4: Clustering

Breve descripción y análisis del dataset

El dataset contiene datos de canciones (tracks) de Spotify. El objetivo es analizar si es posible agrupar los datos de un conjunto de datos en función de algún criterio, mediante el algoritmo K-Means. La cantidad de registros inicial del Dataset fue de 750 filas y 13 columnas.

Como es costumbre, realizamos un breve análisis del Dataset categorizando las columnas en cuantitativas y cualitativas:

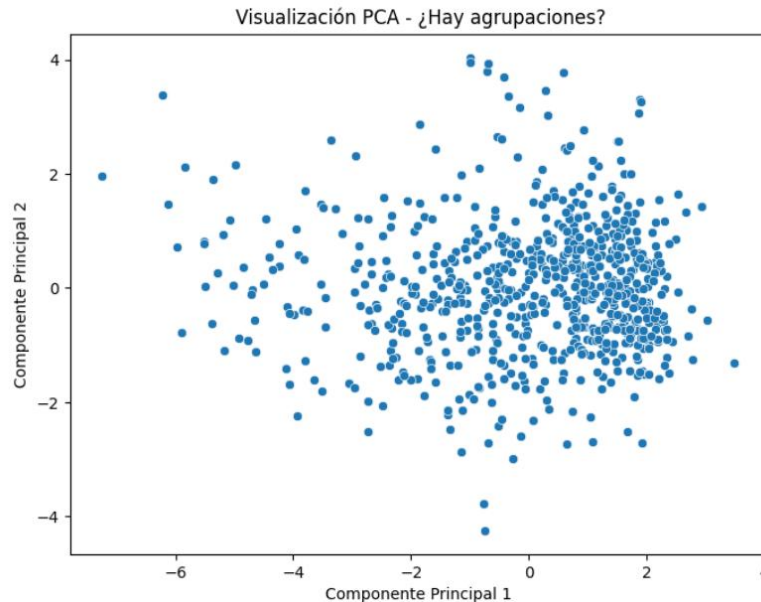
Tipo de variable	Cuantitativas	Tipo de variable	Cualitativas
<u>Continua</u>	acousticness, danceability, duration, energy, instrumentalness, liveness, loudness, speechiness, tempo, valence	<u>Nominal</u>	mode
<u>Discreta</u>	time_signature	<u>Ordinal</u>	key

Tras haberlas categorizado, continuamos con el análisis examinando sus medidas de resumen, frecuencias y los posibles valores que pueden tomar. Luego analizamos los datos faltantes, pero al revisar el dataset descubrimos que no había ninguno. Por lo tanto, no fue necesario imputar valores y procedimos directamente al análisis de outliers.

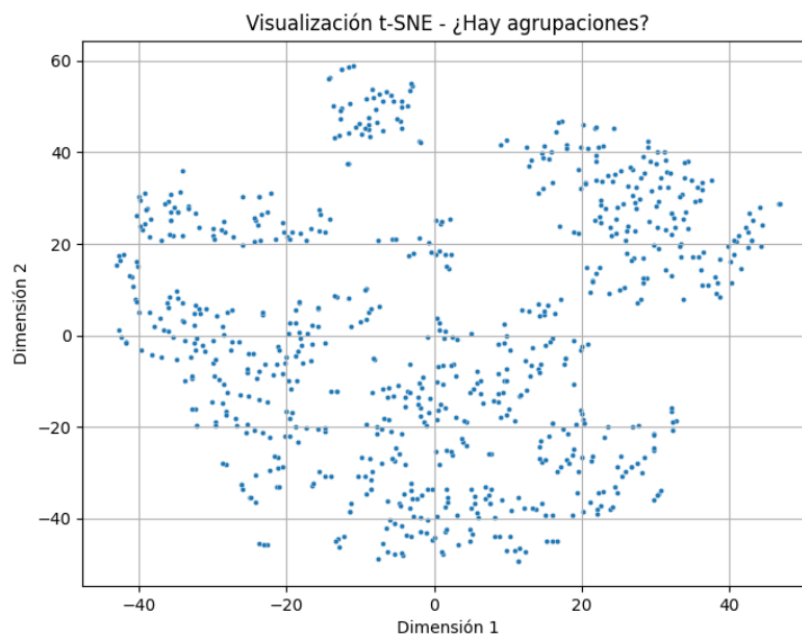
Primero realizamos boxplots para hacer un análisis visual antes de aplicar métodos como el rango intercuartílico (IQR) y la puntuación Z-Score. Al observarlos, notamos que no había valores atípicos especialmente llamativos: en general, los valores identificados como outliers por los boxplots resultaban ser posibles y razonables. El único caso debatible fue el de la variable duration (duración en milisegundos de la canción), donde se observaron pistas con duraciones entre 400.000 y 700.000 ms, equivalentes a aproximadamente 6.67 y 11.67 minutos, respectivamente. Sin embargo, consideramos que estos valores son perfectamente posibles en el contexto musical y no lo suficientemente extremos como para justificar su eliminación o imputación. Quizás si hubieran superado los 15, 20 o 30 minutos, habríamos considerado intervenir.

Análisis de la tendencia al clustering

Una vez finalizado el análisis exploratorio, pasamos al objetivo principal del ejercicio: el clustering. Como primer paso, realizamos un Análisis de Componentes Principales (PCA) con el fin de explorar posibles agrupamientos lineales en los datos. Sin embargo, este análisis no arrojó resultados concluyentes, ya que la proyección en las primeras componentes principales no evidenciaba una separación clara entre grupos.

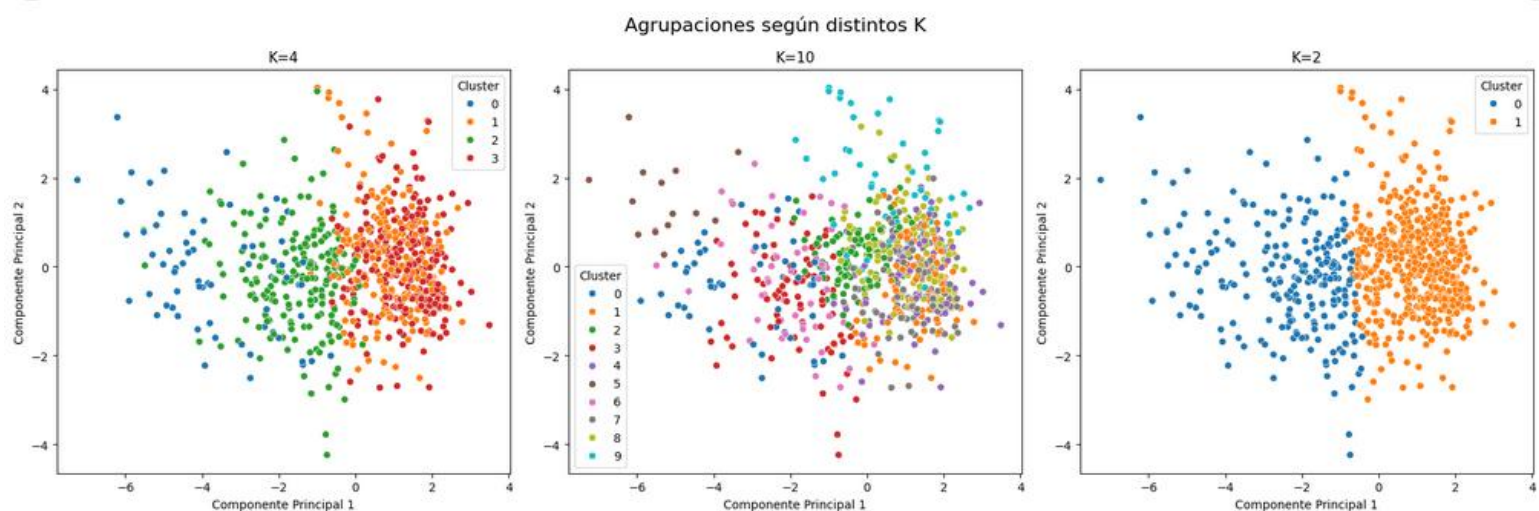


Dado este resultado, optamos por una alternativa no lineal: *t-Distributed Stochastic Neighbor Embedding* (t-SNE). Utilizamos un pipeline para asegurarnos de normalizar adecuadamente los datos antes de aplicar el algoritmo. Tras su implementación, notamos una diferencia considerable con respecto a PCA:



Notamos que la representación resultante mostraba agrupamientos mucho más definidos y visualmente distinguibles, lo que facilitó la identificación de posibles clusters. Con esto en mente, aplicamos el K-Means para hallar los distintos clusters. Probamos con K tomando los valores entre 2 y 10 (inclusivos) mediante el método del Codo y Silhouette. El método del Codo sugirió que 4 era una buena cantidad de grupos. Por este motivo, procedimos a aplicar el algoritmo K-Means con tres valores distintos de k: 2 (como elección aleatoria), 4 (por ser el valor sugerido por el método del codo) y 10 (por representar una de las opciones menos recomendadas, utilizada con fines comparativos). Luego analizamos los gráficos resultantes de cada agrupación para entender cómo se comportaban los datos bajo diferentes configuraciones.

Además, incorporamos las asignaciones de cluster como nuevas columnas en el dataset, lo que hizo que pasara de tener 13 a 16 columnas. Para visualizar los resultados, proyectamos cada una de las agrupaciones sobre el espacio reducido por PCA, y notamos que la opción con $k = 2$ resultaba particularmente interesante, ya que mostraba una mejor adecuación a la estructura de los datos en comparación con las demás alternativas.

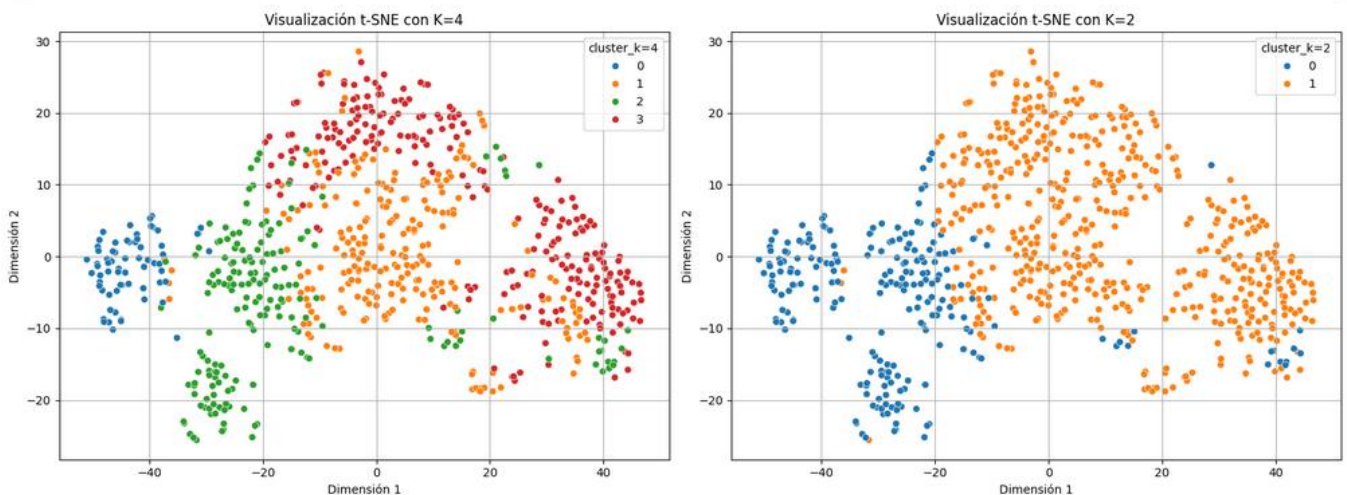


Posteriormente, aplicamos el coeficiente de Silhouette para evaluar la calidad de los agrupamientos obtenidos. Este análisis indicó que la mejor opción era $k=2$, mientras que $k=6$ y $k=8$ obtuvieron el mismo puntaje como las siguientes mejores alternativas. Dado que la agrupación con $k = 10$ había sido incorporada únicamente como referencia y ninguno de los métodos utilizados la consideró adecuada, decidimos eliminar esa columna. En consecuencia, el dataset pasó de tener 16 a 15 columnas.

Con base en toda la información obtenida, decidimos establecer $k = 2$ como el valor óptimo, siguiendo lo indicado por el Silhouette Score. Este coeficiente mide qué tan cerca está un objeto de los demás en su propio cluster en comparación con objetos de otros clusters. Aunque la regla del codo sugería la posibilidad de contar con más clusters ($k = 4$), el análisis de Silhouette respalda la elección de $k = 2$ al obtener mejores métricas de calidad. Este valor permite una clara separación y agrupación de los datos, aspecto fundamental para su interpretación y análisis.

En resumen, consideramos que el número de clusters propuesto por Silhouette es el que realmente logra agrupar de forma coherente, ya que enfatiza la distancia entre clusters y la compacidad interna de los mismos, mientras que la regla del codo se basa únicamente en la inercia (la suma de las distancias cuadráticas dentro de los clusters). No estamos desestimando la utilidad del método del codo, sino que, en este caso, nuestra elección se apoya también en evidencias visuales obtenidas mediante t-SNE y PCA, que reflejaron de manera clara la estructura de los datos para $k = 2$.

Para analizar el agrupamiento utilizamos t-SNE para visualizar los clusters. Graficamos con los dos valores de k para sostener la elección de $k=2$.



Luego aplicamos la media a cada columna del Dataset y utilizamos la librería tabulate para visualizar el promedio de cada variable por cada cluster de forma más ordenada.

cluster_k=2	acousticness	danceability	duration	energy	instrumentalness	key	liveness	loudness	mode	speechiness	tempo	time_signature	valence
0	0.759425	0.466929	232094	0.29864	0.247272	4.36444	0.154536	-13.6362	0.826667	0.0500853	112.451	3.69778	0.366999
1	0.185095	0.651943	214978	0.720851	0.037233	5.02857	0.224308	-6.31212	0.704762	0.119915	123.815	3.99048	0.553174

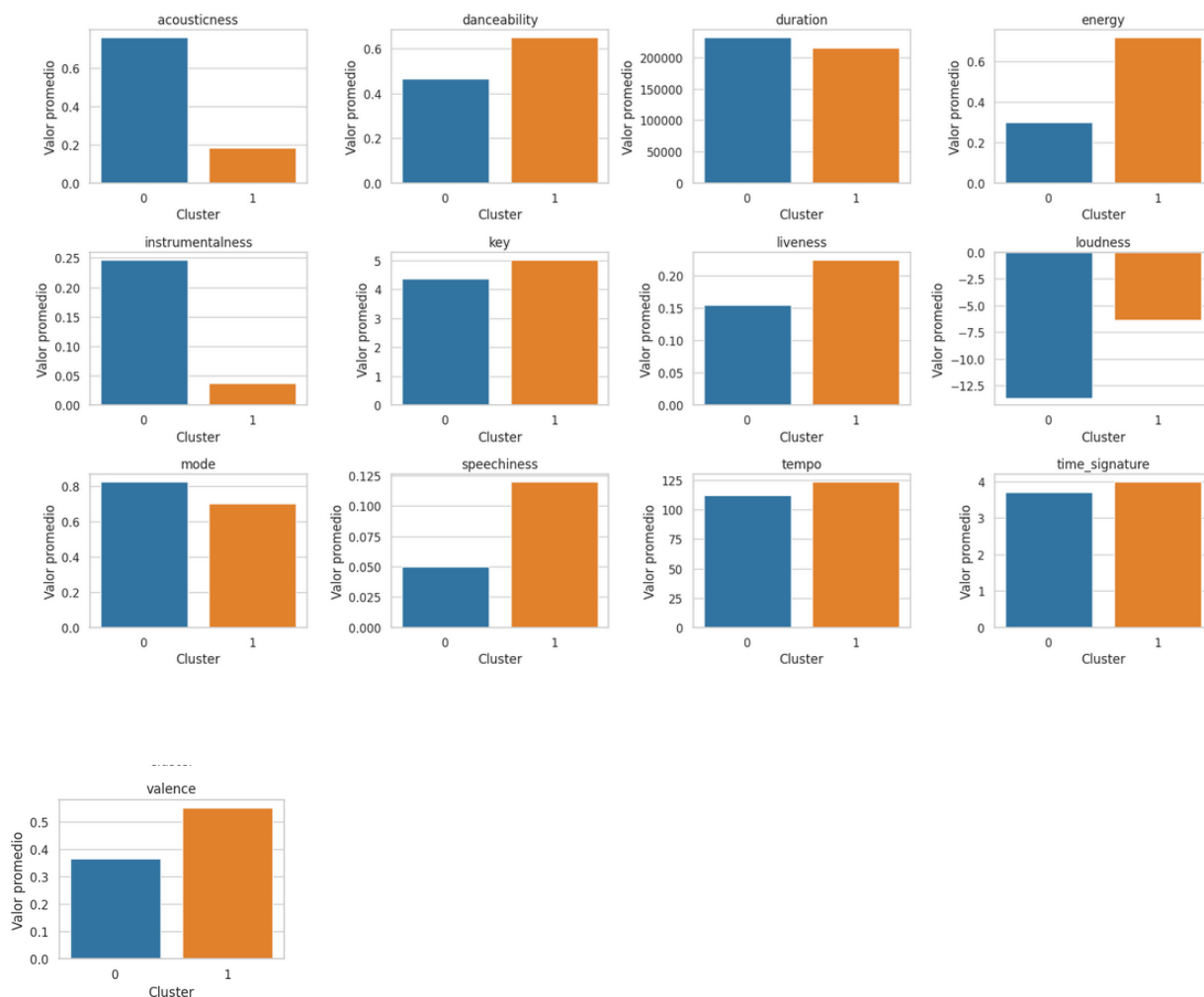
Dada nuestra elección, establecimos que el número de grupos a formar debe ser 2. Estos grupos se caracterizan y dividen de la siguiente manera:

Cluster 0: Variables con valores altos: acousticness (0.76), instrumentalness (0.24) y duration (232094 ms, que equivale a 3.8682 minutos). Estos indican que los tracks agrupados en este cluster tienden a tener más instrumentales, con pocas vocales e incluso ninguna. Además, su duración es la más elevada entre ambas clases. Variables con valores bajos: danceability (0.46), speechiness (0.05), valence (0.36), energy (0.30) y baja loudness (-13.6). Estas características indican que no es música con alto BPM, y que tiende a ser tranquila e incluso melancólica.

Teniendo todo esto en cuenta, más la duración elevada, creemos que agrupó por pistas relacionadas a la música Clásica, la música ambiental, el Jazz suave o los instrumentales. En general, música donde se destaca el instrumento.

Cluster 1: Variables con valores altos: energy (0.72), loudness (-6.3), valence (0.55), danceability (0.65), speechiness (0.12) y tempo (123.8 BPM). Esto indica que son canciones más fuertes, rítmicas y alegres. Son bailables y poseen un BPM mayor. Variables con valores bajos: instrumentalness (0.04) y acousticness (0.19). Esto nos habla de tracks con poco instrumental y con menor acústica. El conjunto de estas características muestra tracks que poseen un alto BPM y se centran en los vocales. Tienden a tener un instrumental básico, electrónico y sencillo y pegadizo. Suelen ser rítmicas y alegres, movidas y bailables. Consideramos que en este grupo se juntan canciones de los géneros Pop, Hip Hop, Rap y Reggaetón.

Además, decidimos implementar unos gráficos de barra que ilustran perfectamente las diferencias de cada cluster respecto a cada variable. Estos permiten comprender la tabla de una forma más sencilla.



En conclusión, nos quedamos con $k=2$ como la cantidad óptima de clusters. Esta elección se basa en que ofrece una separación clara y coherente entre los grupos, lo cual resulta fundamental para la interpretación y el análisis de los datos. Los dos clusters identificados reflejan diferencias marcadas en los géneros musicales: el primer grupo (cluster 0) agrupa principalmente música Clásica, ambiental, Jazz suave e instrumentales, mientras que el segundo (cluster 1) incluye géneros como Pop, Hip Hop, Rap y Reggaetón. Esta división respalda la idea de que existen patrones diferenciados en el dataset según características musicales, y valida la elección de $k=2$ desde un enfoque métrico tanto como interpretativo.

Cuadro de tiempo dedicado

Integrante	Tarea	Prom. Hs Semana
Ramiro Barcala Roca	Armado de gráficos entre distintos K	4

	Implementación del K-Means	
Rafaela Pilar Arias	Análisis exploratorio de las variables Análisis de la tendencia al clustering Realización del informe	5
Mora Lema	Análisis de la tendencia al clustering Análisis Silhouette Realización del informe	5
Lara Eliana Ienco	Análisis de la tendencia al clustering Interpretación de Gráficos	3